

Ifosup Wavre

Conception et développement d'applications

Projet UML

Jeu de quiz
« Projet Cultissime »

Analyse et conception

van der Veen Georgé

Année académique 2025-2026

Table des matières

1	Introduction	5
2	Conventions de notation	5
2.1	Version d'UML retenue	5
2.2	Stéréotypes employés	5
3	Cahier des charges	6
3.1	Contexte et origine de la demande	6
3.2	Destinataires de l'application	7
3.3	Délai attendu	7
3.4	Justification de la demande et analyse concurrentielle	7
3.5	Besoins fonctionnels	8
3.6	Besoins non fonctionnels	9
3.7	Bénéfices attendus	9
3.8	Périmètre du système	10
3.9	Conditions d'utilisation et critères de réussite	10
3.10	Évolutions possibles	11
4	Étude technologique	11
4.1	Contrainte directrice : le temps réel	11
4.2	Solutions back-end envisagées	11
4.3	Solutions front-end envisagées	12
4.4	Base de données	13
5	Diagramme de cas d'utilisation	13
5.1	Frontière du système et acteurs	14
5.2	Cas d'utilisation et relations	15
5.3	Couverture des exigences fonctionnelles	15
5.4	Risque, priorité et planification itérative	16
5.5	Découpage en incréments	17
6	Scénarios des cas d'utilisation	18
6.1	S'authentifier	18
6.2	Rejoindre un salon	19
6.3	Créer un salon	20
6.4	Jouer une partie	20
6.5	Soumettre une question	22
6.6	Souscrire un abonnement	22
7	Diagrammes de séquence système (boîte noire)	23
7.1	S'authentifier	24
7.2	Rejoindre un salon	25
7.3	Créer un salon	26
7.4	Jouer une partie	27
7.5	Soumettre une question	28
7.6	Souscrire un abonnement	29

8 Diagrammes de séquence de conception	30
8.1 Jouer une partie	31
8.2 S'authentifier	32
8.3 Rejoindre un salon	33
8.4 Créer un salon	34
8.5 Soumettre une question	34
8.6 Souscrire un abonnement	35
9 Diagramme de classes	36
9.1 Choix de modélisation	36
10 Diagrammes de collaboration	37
10.1 Niveau système (boîte noire)	37
10.2 Niveau de conception (boîte blanche)	41
11 Diagramme d'activité	43
12 Diagramme d'états-transitions	44
13 Diagramme de packages	44
14 Traduction des classes en code	45
15 Glossaire	47
16 Annexe : diagramme de classes en pleine page	50

Table des figures

Fig. 1 Diagramme de cas d'utilisation du système « Projet Cultissime »	14
Fig. 2 Séquence système — « S'authentifier » (scénario nominal)	24
Fig. 3 Séquence système — « S'authentifier » (scénario d'erreur E1 : identifiants incorrects)	24
Fig. 4 Séquence système — « Rejoindre un salon » (scénario nominal)	25
Fig. 5 Séquence système — « Rejoindre un salon » (scénario alternatif A1 : salon privé)	25
Fig. 6 Séquence système — « Créer un salon » (scénario nominal)	26
Fig. 7 Séquence système — « Créer un salon » (scénario d'erreur E2 : configuration incohérente)	26
Fig. 8 Séquence système — « Jouer une partie » (scénario nominal)	27
Fig. 9 Séquence système — « Jouer une partie » (scénario d'erreur E3 : temps écoulé sans réponse)	28
Fig. 10 Séquence système — « Soumettre une question » (scénario nominal)	28

Fig. 11 Séquence système — « Soumettre une question » (scénario d'erreur E1 : champ obligatoire manquant).	29
Fig. 12 Séquence système — « Souscrire un abonnement » (scénario nominal).	29
Fig. 13 Séquence système — « Souscrire un abonnement » (scénario d'erreur E1 : paiement refusé).	30
Fig. 14 Séquence de conception — « Jouer une partie » (scénario nominal).	31
Fig. 15 Séquence de conception — « Jouer une partie » (E3 : temps écoulé, message dérobant).	31
Fig. 16 Séquence de conception — « S'authentifier » (scénario nominal).	32
Fig. 17 Séquence de conception — « S'authentifier » (E1 : identifiants incorrects).	32
Fig. 18 Séquence de conception — « Rejoindre un salon » (scénario nominal).	33
Fig. 19 Séquence de conception — « Rejoindre un salon » (A1 : salon privé).	33
Fig. 20 Séquence de conception — « Créer un salon » (scénario nominal).	34
Fig. 21 Séquence de conception — « Créer un salon » (E2 : configuration incohérente).	34
Fig. 22 Séquence de conception — « Soumettre une question » (scénario nominal).	34
Fig. 23 Séquence de conception — « Soumettre une question » (E1 : champ obligatoire manquant).	35
Fig. 24 Séquence de conception — « Souscrire un abonnement » (scénario nominal).	35
Fig. 25 Séquence de conception — « Souscrire un abonnement » (E1 : paiement refusé).	35
Fig. 26 Diagramme de classes de conception du système « Projet Cultissime ».	36
Fig. 27 Collaboration système — « S'authentifier ».	37
Fig. 28 Collaboration système — « S'authentifier » (E1 : identifiants incorrects).	38
Fig. 29 Collaboration système — « Rejoindre un salon ».	38
Fig. 30 Collaboration système — « Rejoindre un salon » (A1 : salon privé).	38
Fig. 31 Collaboration système — « Créer un salon ».	39

Fig. 32	Collaboration système — « Jouer une partie ».	39
Fig. 33	Collaboration système — « Jouer une partie » (E3 : temps écoulé).	39
Fig. 34	Collaboration système — « Créer un salon » (E2 : configuration incohérente).	40
Fig. 35	Collaboration système — « Soumettre une question ».	40
Fig. 36	Collaboration système — « Soumettre une question » (E1 : champ manquant).	40
Fig. 37	Collaboration système — « Souscrire un abonnement ».	40
Fig. 38	Collaboration système — « Souscrire un abonnement » (E1 : paiement refusé).	41
Fig. 39	Collaboration de conception — « S'authentifier ».	41
Fig. 40	Collaboration de conception — « S'authentifier » (E1 : identifiants incorrects).	41
Fig. 41	Collaboration de conception — « Rejoindre un salon ».	41
Fig. 42	Collaboration de conception — « Rejoindre un salon » (A1 : salon privé).	41
Fig. 43	Collaboration de conception — « Créer un salon ».	41
Fig. 44	Collaboration de conception — « Créer un salon » (E2 : configuration incohérente).	41
Fig. 45	Collaboration de conception — « Jouer une partie ».	42
Fig. 46	Collaboration de conception — « Jouer une partie » (E3 : temps écoulé, message dérobant).	42
Fig. 47	Collaboration de conception — « Soumettre une question ».	42
Fig. 48	Collaboration de conception — « Soumettre une question » (E1 : champ manquant).	42
Fig. 49	Collaboration de conception — « Souscrire un abonnement ».	42
Fig. 50	Collaboration de conception — « Souscrire un abonnement » (E1 : paiement refusé).	42
Fig. 51	Diagramme d'activité du déroulement d'une partie (couloirs Joueur / Système).	43
Fig. 52	Diagramme d'états-transitions du cycle de vie d'une question.	44
Fig. 53	Diagramme de packages du système « Projet Cultissime ».	45
Fig. 54	Diagramme de classes de conception — version agrandie en pleine page.	50

1 Introduction

Le présent document constitue l'analyse et la conception d'une application web de quiz multijoueur, baptisée « Projet Cultissime » pour le moment, réalisée dans le cadre de l'épreuve intégrée. Il a pour objet d'étudier les fonctionnalités attendues du système au travers des différents diagrammes de l'uml, selon les visions fonctionnelle, dynamique et statique.

L'application vise à proposer un jeu de quiz rapide en temps réel, où des joueurs réunis dans un même salon répondent simultanément à des questions, dans l'esprit de jeux existants comme PopSauce, mais en corrigeant plusieurs de leurs limites. Le projet met l'accent sur l'accès immédiat au jeu, la richesse et la qualité du contenu, et une expérience soignée.

Le document s'ouvre sur le *cahier des charges*, qui recense les besoins et délimite le périmètre du système. Il se poursuit par le *diagramme de cas d'utilisation*, qui en formalise la vision fonctionnelle, accompagné de la couverture des exigences et du découpage du développement en incréments. Les diagrammes dynamiques et statiques, ainsi que le glossaire, complètent l'analyse. Conformément aux consignes, le cahier des charges pourra faire l'objet d'amendements au fil de l'analyse.

2 Conventions de notation

Afin de garantir la cohérence et la lisibilité de l'analyse, les diagrammes de ce document suivent un ensemble de conventions explicitées ici.

2.1 Version d'UML retenue

L'analyse est menée en notation uml 1.4.1. Certaines constructions n'existent toutefois que dans des versions ultérieures de la norme ; lorsqu'une telle construction est employée, elle est signalée par le stéréotype « UML 2.0 », afin que le lecteur identifie clairement l'emprunt. C'est notamment le cas des fragments combinés (boucle, alternative) des diagrammes de séquence. Pour les scénarios alternatifs et d'erreur, la représentation par diagrammes séparés — native en UML 1.4.1 — a été préférée aux fragments combinés. Les diagrammes de classes, d'états-transitions, de packages et d'activité présentés ici n'emploient aucune construction de ce type et relèvent donc intégralement d'UML 1.4.1.

2.2 Stéréotypes employés

Les stéréotypes, notés entre guillemets français « ... », précisent la nature d'un élément. Le tableau suivant récapitule ceux utilisés dans le document.

Stéréotype	Diagramme(s)	Signification
« include »	Cas d'utilisation	Le cas de base intègre systématiquement le comportement du cas inclus.
« extend »	Cas d'utilisation	Le cas d'extension complète le cas de base de façon optionnelle, sous condition.

Stéréotype	Diagramme(s)	Signification
« boundary »	Séquence (conception)	Objet d'interface (frontière) de Jacobson : point de contact entre un acteur et le système.
« control »	Séquence (conception)	Objet de contrôle de Jacobson : orchestre la logique applicative d'un cas d'utilisation.
« entity »	Séquence (conception)	Objet entité de Jacobson : donnée métier persistante.
« create »	Séquence, collaboration	Message de création d'un objet.
« destroy »	Séquence, collaboration	Message de destruction d'un objet.
« minuté »	Séquence, collaboration	Message soumis à une contrainte de temps (décompte du chronomètre).
« dérobant »	Séquence, collaboration	Message qui n'aboutit que si le récepteur est en mesure de le traiter, et est abandonné sinon (ici, une réponse arrivant après l'échéance du chronomètre).
« enumeration »	Classes	Classificateur dont les instances sont un ensemble fini de valeurs nommées.
« use »	Packages	Dépendance d'utilisation : un paquet dépend des éléments d'un autre.
« UML 2.0 »	Tous	Marque une construction empruntée à UML 2.0 dans une analyse menée en UML 1.4.1.

3 Cahier des charges

3.1 Contexte et origine de la demande

Ce projet émane d'un constat personnel : un manque dans l'offre actuelle de jeux de quiz. La demande n'a donc pas d'origine externe ; c'est l'auteur lui-même qui, à partir de sa propre expérience de joueur, est à l'origine de la démarche. Le projet prend pour référence principale le jeu en ligne PopSauce, dont le principe consiste à rejoindre un salon et à répondre le plus rapidement possible à des questions de culture populaire : chaque question rapporte par défaut dix points au joueur le plus rapide, et la victoire est attribuée au premier à franchir le seuil de cent points.

L'objectif n'est pas de reproduire PopSauce à l'identique, mais d'en proposer une version enrichie qui corrige plusieurs limites observées sur les plateformes existantes.

3.2 Destinataires de l'application

L'application est destinée à un large public d'amateurs de jeux de connaissances et de rapidité. On distingue principalement :

- les **joueurs occasionnels**, qui rejoignent une partie publique ponctuellement, sans nécessairement créer de compte ;
- les **joueurs réguliers**, qui souhaitent suivre leurs statistiques et leur progression dans le temps ;
- les **communautés en ligne** (groupes d'amis, serveurs Discord, communautés de streaming) qui organisent des parties privées ;
- les **créateurs de contenu**, qui conçoivent et partagent leurs propres packs de questions.

3.3 Délai attendu

L'application est attendue pour la fin de l'année académique 2026-2027 dans le cadre du travail de fin d'études. Le développement suivra une démarche itérative et incrémentale, permettant de livrer d'abord un noyau jouable (un mode de jeu fonctionnel en multi-joueur), puis d'enrichir progressivement la plateforme avec les fonctionnalités secondaires (ajout de questions, modes de jeu additionnels, modèle économique, etc.). Le découpage détaillé en incréments est présenté à la suite du diagramme de cas d'utilisation.

3.4 Justification de la demande et analyse concurrentielle

La demande est motivée par le constat que les plateformes de quiz rapides existantes restent peu abouties. Les principaux concurrents identifiés et les limites relevées sont les suivants :

Concurrent	Limites observées
PopSauce	Interface peu ergonomique et datée ; questions très orientées culture populaire, avec parfois un manque de cohérence ou de catégorisation.
Kculture	Modèle économique contraignant (abonnement à une chaîne Twitch) ; réponses validées par l'hôte en fin de partie plutôt qu'en temps réel ; catalogue de questions limité.
Zequestion	Plateforme peu connue ; absence de développement et d'animation autour du jeu.

Ce panorama fait apparaître des marges d'amélioration claires : l'ergonomie, la richesse et la cohérence du contenu, la souplesse des règles de cotation, et un modèle économique mieux équilibré.

3.5 Besoins fonctionnels

Le système doit répondre aux besoins fonctionnels suivants.

Jeu multijoueur en temps réel. Le cœur de l'application est la partie multijoueur synchrone : plusieurs joueurs réunis dans un même salon répondent simultanément à une série de questions, et un classement en direct reflète la rapidité et l'exactitude de leurs réponses. Le système doit gérer la création de salons publics et privés, la synchronisation des questions entre tous les participants et le décompte du temps.

Modes de jeu variés. Au-delà de la course au score classique, le système proposera plusieurs modes (par exemple : élimination progressive par système de vies, jeu en équipes, etc.). Chaque salon pourra choisir son mode au lancement de la partie. Le type de cotation sera également configurable : soit au temps, à la manière de PopSauce, soit par validation en fin de partie, à la manière de Kculture, cette seconde option offrant davantage de souplesse sur l'orthographe des réponses.

Gestion du salon par l'hôte. Le joueur ayant créé un salon en est l'hôte : il peut, au-delà de la configuration, gérer le déroulement de la partie en excluant un joueur perturbateur et en fermant le salon. Cette capacité de modération en cours de partie est importante dans un contexte où des invités anonymes peuvent rejoindre les salons publics.

Contenu mixte : officiel et communautaire. Le catalogue de questions reposera sur deux sources. Un ensemble de packs **officiels** sera fourni et maintenu par l'administrateur de la plateforme. En parallèle, les utilisateurs pourront créer et partager leurs propres **packs communautaires**. Pour garantir la qualité du contenu, ces contributions passeront par une file de modération avant publication. En complément, un mécanisme de vote positif/négatif à chaque question permettra de remonter les contenus problématiques et de trier automatiquement le contenu : les questions les mieux notées sont présentées plus fréquemment, les questions les moins bien notées le sont moins. Une question accumulant un taux de votes négatifs élevé sera automatiquement renvoyée en file de modération pour réexamen.

Catégorisation des questions. Les questions seront classées au moyen d'un système de tags fournis. Une assistance par intelligence artificielle pourra être envisagée pour suggérer automatiquement les catégories pertinentes lors de la création d'une question.

Modération assistée. La vérification des questions soumises par la communauté pourra elle aussi s'appuyer sur une assistance par intelligence artificielle, afin de présélectionner les contenus à valider et d'alléger le travail de l'administrateur.

Profil et statistiques. Les joueurs disposant d'un compte pourront consulter un profil personnel regroupant leur historique de parties et leurs statistiques (parties jouées, taux de bonnes réponses, temps de réponse moyen, packs favoris).

Accès souple. L'authentification ne sera pas obligatoire : un utilisateur pourra rejoindre une partie en tant qu'invité, sans inscription. La création d'un compte restera optionnelle et déblocuera les fonctionnalités persistantes (profil, statistiques, création et sauvegarde de packs). Un mécanisme de récupération de mot de passe permettra à un utilisateur authentifié de retrouver l'accès à son compte.

Modèle économique. Un nombre restreint de questions officielles (de l'ordre de vingt à quarante) sera accessible gratuitement chaque jour. Le créateur d'un salon pourra souscrire un abonnement donnant accès à l'intégralité du catalogue officiel. Afin d'éviter les doublons, certaines catégories sensibles (par exemple « deviner le pays ») seront réservées au contenu officiel et donc accessibles uniquement via l'abonnement, plutôt que reproduites dans les packs communautaires. Les abonnés bénéficieront en outre d'éléments cosmétiques (skins d'avatar, thèmes d'interface).

3.6 Besoins non fonctionnels

- **Performance et équité** : le jeu reposant sur la rapidité, la synchronisation des questions entre les joueurs d'un même salon doit être quasi instantanée — une latence de l'ordre de quelques centaines de millisecondes au maximum — afin qu'aucun joueur ne soit avantagé par sa connexion.
- **Montée en charge** : le système doit supporter plusieurs salons actifs en parallèle et un nombre raisonnable de joueurs simultanés par salon (de l'ordre de la dizaine), sans dégradation perceptible.
- **Disponibilité** : l'application doit rester accessible de manière fiable, les interruptions en cours de partie étant particulièrement préjudiciables à l'expérience.
- **Ergonomie** : l'interface doit être soignée et agréable, en réponse directe au principal reproche fait aux plateformes concurrentes.
- **Compatibilité** : l'application doit rester utilisable confortablement sur les navigateurs web courants et récents.
- **Évolutivité** : l'architecture doit permettre un portage ultérieur (mobile ou application de bureau) et l'ajout de modes de jeu sans refonte majeure.
- **Sécurité et données personnelles** : les comptes et les statistiques constituant des données personnelles, leur traitement devra respecter la réglementation applicable (RGPD) ; les mots de passe seront stockés de façon sécurisée.
- **Qualité du contenu** : le contenu communautaire doit être modéré avant publication, puis trié en continu par le vote des joueurs.

Sur le plan des contraintes techniques, le caractère temps réel du jeu oriente vers une architecture client web communiquant avec un serveur au moyen d'une connexion persistante (par exemple via des WebSockets), adossé à une base de données pour les questions, les comptes et les statistiques. Le choix définitif des technologies n'est pas figé à ce stade de l'analyse ; les solutions envisagées sont comparées dans l'étude technologique présentée au chapitre suivant.

3.7 Bénéfices attendus

Les bénéfices visés sont les suivants :

- une **expérience de jeu plus riche et rejouable**, grâce à la diversité des modes et au renouvellement permanent du contenu communautaire ;
- une **barrière d'entrée faible**, l'accès en mode invité permettant de jouer immédiatement ;
- une **fidélisation des joueurs**, par le suivi de leur progression et la reconnaissance de leurs contributions ;

- une **plateforme évolutive**, dont le catalogue s'enrichit par la communauté sans intervention manuelle constante de l'administrateur ;
- un **modèle économique soutenable**, fondé sur l'abonnement et les éléments cosmétiques plutôt que sur la publicité intrusive.

3.8 Périmètre du système

Le périmètre du système, pour cette première version, est le suivant :

- **Plateforme** : application **web** uniquement dans un premier temps. L'architecture sera néanmoins pensée pour permettre un portage ultérieur (mobile ou application de bureau) sans refonte majeure.
- **Portée fonctionnelle** : le système couvre l'ensemble de la chaîne de jeu (création de salon, déroulement d'une partie, classement), la gestion du contenu (création, catégorisation, modération, vote), la gestion des profils ainsi que la gestion des abonnements.
- **Hors périmètre** : pour cette première version sont explicitement exclus les applications mobiles et de bureau (le portage est anticipé mais non réalisé), l'intégration d'un paiement réel (le mécanisme d'abonnement pourra être simulé dans le cadre de l'épreuve), ainsi que toute fonctionnalité de réseau social avancé (messagerie privée, fil d'actualité, système d'amis).

Trois acteurs sont identifiés à ce stade :

- le **Visiteur** : utilisateur non authentifié, qui peut rejoindre et jouer une partie en tant qu'invité. Il peut également créer et configurer un salon, mais de façon limitée (par exemple : pas d'accès aux questions officielles réservées à l'abonnement, pas de sauvegarde de la configuration du salon entre deux sessions) ;
- l'**Utilisateur authentifié** : il dispose de toutes les possibilités du visiteur, sans les limitations de ce dernier, auxquelles s'ajoutent le suivi de son profil et de ses statistiques ainsi que la soumission de questions à la communauté ;
- l'**Administrateur** : il gère la plateforme et valide (ou refuse) les questions soumises par les utilisateurs avant leur publication.

Ces rôles sont précisés et formalisés dans le diagramme de cas d'utilisation.

3.9 Conditions d'utilisation et critères de réussite

L'objectif sera considéré comme atteint si le système satisfait les conditions suivantes :

- une partie multijoueur peut être lancée et menée à son terme par plusieurs joueurs simultanés sans désynchronisation perceptible ;
- un utilisateur peut rejoindre et jouer une partie en mode invité, sans inscription préalable ;
- un utilisateur peut soumettre un pack de questions, ce pack n'étant rendu jouable qu'après validation par la modération ;
- un joueur authentifié retrouve, d'une session à l'autre, son profil et ses statistiques à jour ;
- un mode de jeu est pleinement fonctionnel, l'architecture étant conçue pour permettre l'ajout d'autres modes sans refonte ;
- les deux types de cotation (au temps et par validation en fin de partie) sont opérationnels.

3.10 Évolutions possibles

Au-delà du périmètre retenu, plusieurs fonctionnalités ont été identifiées comme des évolutions envisageables pour une version ultérieure. Elles sont écartées de cette première version afin de concentrer l'effort sur le cœur de l'expérience, mais sont mentionnées ici pour situer les perspectives du projet :

- **fin de partie enrichie** : revanche immédiate dans le même salon, partage du résultat, retour au lobby ;
- **mini-chat de salon** : messagerie légère limitée à la partie en cours, pour la convivialité ;
- **édition d'un pack soumis** par son créateur, avec re-modération, afin de corriger une question erronée ;
- **page de découverte** mettant en avant les packs communautaires les mieux notés ;
- **classements globaux** (ligues, saisons) et fonctionnalités sociales (système d'amis), volontairement exclus à ce stade en raison de leur complexité ;
- **portage mobile ou application de bureau**, anticipé par l'architecture mais non réalisé.

4 Étude technologique

Le choix des technologies n'est pas figé à ce stade. Cette section recense les solutions envisagées et les met en regard de la contrainte directrice du projet, afin d'éclairer une décision ultérieure.

4.1 Contrainte directrice : le temps réel

La fonctionnalité déterminante de l'application est la partie multijoueur synchrone : une même question est diffusée simultanément à tous les joueurs d'un salon, un chronomètre est partagé et un classement se met à jour en direct. Cela impose une communication bidirectionnelle persistante entre le serveur et les clients, typiquement au moyen de *WebSockets*, là où une application web classique se contente de requêtes-réponses. Les autres fonctionnalités (comptes, packs, modération, abonnements) relèvent en revanche d'un développement web classique que toutes les solutions étudiées prennent en charge sans difficulté. C'est donc la qualité du support temps réel qui distingue principalement les options.

4.2 Solutions back-end envisagées

ASP.NET Core avec SignalR (C#). SignalR est une bibliothèque de communication temps réel qui abstrait la gestion des *WebSockets* (reconnexion automatique, repli sur d'autres transports, diffusion groupée). Sa notion de *groupes* correspond directement à celle de salon : ajouter ou retirer un joueur d'un groupe, puis diffuser un message à l'ensemble du groupe, se fait nativement. C'est l'une des solutions les plus matures du marché pour ce besoin précis. Le diagramme de classes de conception se traduit par ailleurs naturellement en C#.

Python avec FastAPI. FastAPI est un cadriciel web asynchrone moderne, doté d'un support natif des *WebSockets* et reconnu pour sa rapidité de développement. La gestion des salons et de la diffusion doit être implémentée explicitement, et la montée en charge sur

plusieurs processus s'appuie généralement sur un mécanisme de publication-souscription externe (par exemple Redis). Le langage est d'un abord aisé et favorise une mise au point rapide de la logique de jeu.

Rust avec Axum ou Actix-web. Rust offre des performances et une maîtrise de la latence et de la concurrence particulièrement adaptées à un serveur temps réel, au prix d'une courbe d'apprentissage sensiblement plus élevée (gestion de la propriété, programmation asynchrone). C'est la solution la plus exigeante, mais aussi la plus formatrice et la plus performante.

Critère	Atouts	Limites
ASP.NET + SignalR	Temps réel le plus abouti (groupes = salons) ; intégration front/back forte ; écosystème mûr.	Langage et environnement à reprendre en main ; pile propriétaire à l'origine.
Python (FastAPI)	Développement rapide ; asynchrone ; langage maîtrisé ; large écosystème.	Gestion des salons et de la diffusion à câbler ; montée en charge à outiller.
Rust (Axum/Actix)	Performances et latence excellentes ; sûreté mémoire ; valeur d'apprentissage.	Courbe d'apprentissage forte ; productivité initiale réduite ; risque sur les délais.

4.3 Solutions front-end envisagées

Blazor. Cadriciel d'interface en C#, il s'intègre étroitement à un back-end ASP.NET et repose lui-même sur SignalR pour son mode serveur, ce qui en fait un complément cohérent de la solution .NET. Il évite d'écrire du JavaScript.

Vue. Cadriciel JavaScript réputé pour sa douceur de prise en main et sa clarté. Il convient bien à une interface réactive (chronomètre, classement en direct) et s'associe à n'importe quel back-end via WebSockets.

React. Cadriciel JavaScript très répandu, doté d'un vaste écosystème. Plus verbeux que Vue, il offre en contrepartie une grande richesse de bibliothèques et une forte demande sur le marché.

Front-end	Atouts	Limites
Blazor	Intégration native avec .NET ; pas de JavaScript ; même langage que le back-end C#.	Couplage fort à l'écosystème .NET ; moins pertinent hors back-end .NET.
Vue	Prise en main aisée ; réactivité ; indépendant du back-end.	Écosystème plus restreint que React.
React	Écosystème très riche ; largement répandu.	Plus verbeux ; davantage de configuration.

On notera la cohérence particulière du couple ASP.NET + Blazor (même langage de part et d'autre), tandis que Vue et React s'associent indifféremment à un back-end Python, Rust ou .NET.

4.4 Base de données

Les données du système sont fortement structurées et reliées entre elles, comme le montrera le diagramme de classes : un joueur participe à des parties, un pack contient des questions, une question admet des réponses, etc. Cette nature relationnelle oriente naturellement vers un *système de gestion de base de données relationnelle* (SGBDR), où chaque entité devient une table et où les associations se traduisent par des clés étrangères — l'association plusieurs-à-plusieurs entre question et tag donnant lieu, à ce niveau, à une table de jointure.

PostgreSQL constitue un choix de référence : libre, robuste, riche en fonctionnalités et compatible avec les trois back-ends envisagés (via Entity Framework pour .NET, SQLAlchemy pour Python, Diesel ou SeaORM pour Rust). **MySQL** ou **MariaDB** représentent des alternatives équivalentes pour ce projet. **SQL Server** s'intègre naturellement à l'écosystème .NET mais reste propriétaire.

En complément du SGBDR, l'état éphémère d'une partie en cours (joueurs connectés, scores instantanés, file de diffusion) gagne à être géré par un magasin de données en mémoire tel que **Redis**, qui sert également de mécanisme de publication-souscription pour synchroniser plusieurs instances du serveur temps réel. Une base orientée document (par exemple MongoDB) a été écartée : la structure des données étant nettement relationnelle, elle n'apporterait pas d'avantage déterminant ici.

En résumé, la persistance reposerait sur un SGBDR (PostgreSQL par défaut) pour les données durables, éventuellement secondé par Redis pour l'état temps réel — ce schéma restant valable quel que soit le back-end finalement retenu.

5 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation formalise la vision fonctionnelle du système : il délimite la frontière de l'application, identifie les acteurs qui interagissent avec elle et recense les services qu'elle leur rend. La notation employée est conforme à UML 1.4.1.

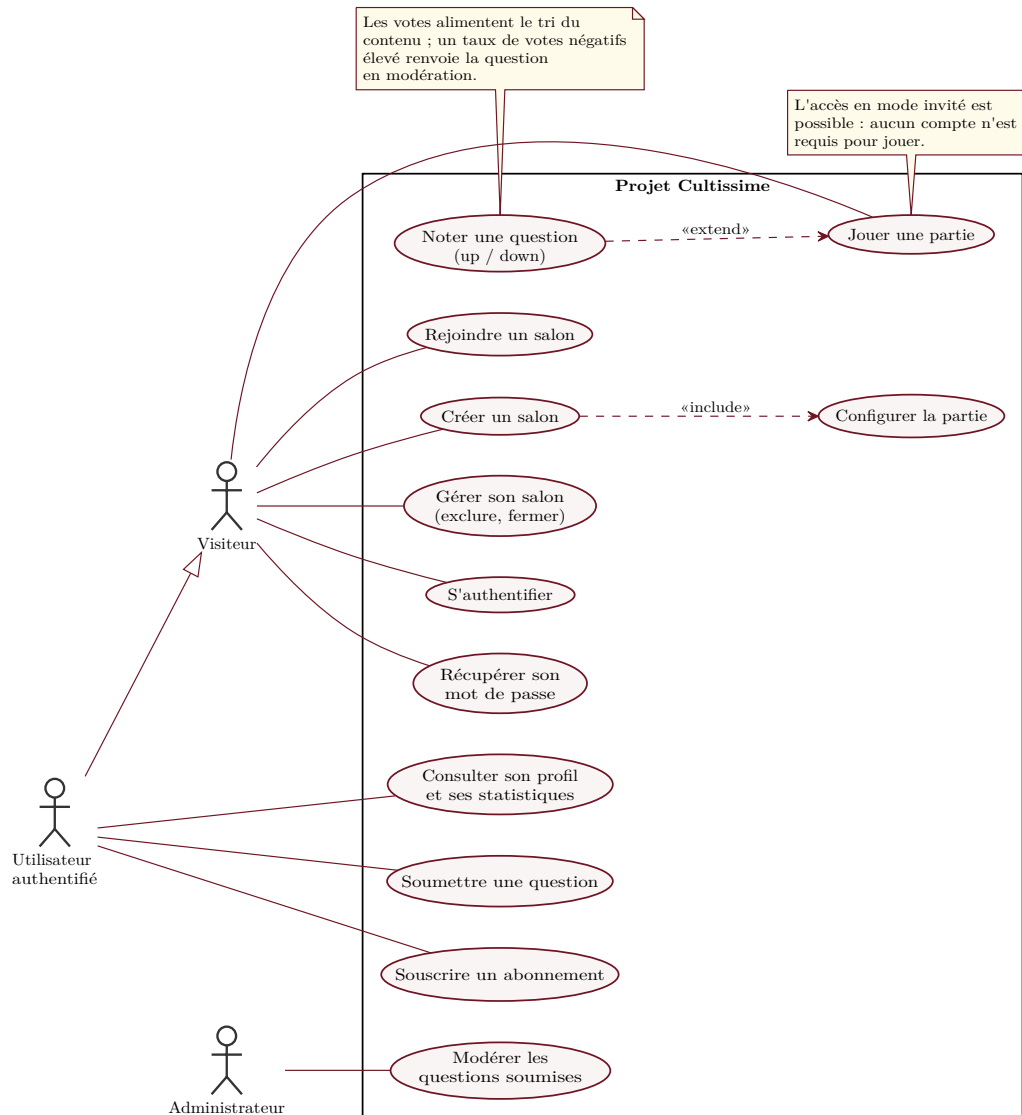


Fig. 1. – Diagramme de cas d'utilisation du système « Projet Cultissime ».

5.1 Frontière du système et acteurs

La frontière du système est représentée par le rectangle englobant l'ensemble des cas d'utilisation ; tout ce qui se trouve à l'extérieur (les acteurs) ne fait pas partie du système mais interagit avec lui. Les trois acteurs identifiés sont tous des acteurs *primaires* : chacun déclenche lui-même les cas d'utilisation auxquels il est relié et en retire un bénéfice direct.

Une relation de *généralisation* relie l'**Utilisateur authentifié** au **Visiteur** : l'utilisateur authentifié est un visiteur particulier. Il hérite donc de tous les cas d'utilisation du visiteur (s'authentifier, rejoindre un salon, créer un salon, jouer une partie) auxquels s'ajoutent ses propres cas (consulter son profil, soumettre une question, souscrire un abonnement). Cet héritage évite de redessiner les associations communes.

À ce stade de l'analyse, le système est volontairement considéré comme une *boîte noire* : seuls les services rendus aux acteurs sont décrits, sans préjuger de la structure interne qui les réalisera. Le diagramme ne comporte donc aucun acteur secondaire, aucun acteur de type matériel externe, ni aucun autre système considéré comme acteur. Les services externes que la plateforme sollicitera par la suite — un prestataire de paiement pour

les abonnements, un service d'intelligence artificielle pour la catégorisation et l'aide à la modération — sont, à ce niveau d'abstraction, traités comme des mécanismes internes et n'apparaissent pas comme acteurs ; ils pourront être explicités lors de la phase de conception. Enfin, conformément aux consignes du cours, la notation graphique retenue est celle d'uml 1.4.1.

5.2 Cas d'utilisation et relations

Les cas d'utilisation se répartissent selon les acteurs de la manière suivante :

- le **Visiteur** peut *s'authentifier, récupérer son mot de passe, rejoindre un salon, créer un salon, gérer son salon* (en exclure un joueur, le fermer) et *jouer une partie* ;
- l'**Utilisateur authentifié** dispose en plus de *consulter son profil et ses statistiques, soumettre une question* et *souscrire un abonnement* ;
- l'**Administrateur** est responsable de *modérer les questions soumises*.

Deux types de relations entre cas d'utilisation apparaissent dans le diagramme.

La relation `<<include>>` traduit une dépendance obligatoire : le cas de base intègre systématiquement le cas inclus. Ici, *créer un salon* inclut toujours *configurer la partie* (choix du mode de jeu, du type de cotation, du caractère public ou privé du salon) : on ne crée pas de salon sans le configurer. La flèche pointe du cas de base vers le cas inclus.

La relation `<<extend>>` traduit un comportement optionnel : le cas d'extension ajoute, sous condition, un comportement au cas de base. Ici, *noter une question (up/down)* étend *jouer une partie* : à l'issue de chaque question, le joueur peut — sans y être obligé — attribuer un vote positif ou négatif. Ce vote alimente le tri automatique du contenu décrit dans le cahier des charges. La flèche pointe du cas d'extension vers le cas de base.

L'acte de répondre à une question n'apparaît pas comme un cas distinct : il constitue le déroulement même de *jouer une partie* et sera détaillé dans le scénario nominal correspondant, avec ses enchaînements alternatifs (bonne réponse, mauvaise réponse, temps écoulé).

5.3 Couverture des exigences fonctionnelles

Afin de vérifier que les besoins fonctionnels sont bien pris en charge, le tableau suivant croise les principaux scénarios des cas d'utilisation avec quatre exigences fonctionnelles clés. Une marque indique qu'un scénario contribue à la satisfaction de l'exigence concernée. On retient :

- **E1** — partie multijoueur en temps réel ;
- **E2** — gestion du contenu (officiel et communautaire) ;
- **E3** — accès souple (mode invité ou compte) ;
- **E4** — modèle économique (abonnement).

Cas d'utilisation	Scénario	E1	E2	E3	E4
Jouer une partie	Nominal — déroulement d'une partie	•			
	Alternatif — noter une question (up/down)		•		

Cas d'utilisation	Scénario	E1	E2	E3	E4
Créer un salon	Erreur — déconnexion d'un joueur	•			
	Nominal — salon public configuré	•			
	Alternatif — salon privé	•		•	
	Alternatif — sélection de questions officielles		•		•
Rejoindre un salon	Nominal — en tant qu'invité	•		•	
	Alternatif — en tant qu'authentifé			•	
Soumettre une question	Nominal — soumission acceptée		•		
	Erreur — rejet par la modération		•		
Souscrire un abonnement	Nominal — paiement accepté				•
	Erreur — paiement refusé				•

Chaque exigence est couverte par au moins un scénario, ce qui confirme que les cas d'utilisation identifiés traitent l'ensemble des fonctionnalités attendues du système.

5.4 Risque, priorité et planification itérative

Le développement s'inscrivant dans une démarche itérative et incrémentale, chaque cas d'utilisation est évalué selon deux axes. Le *risque* (haut, moyen, bas) traduit la difficulté technique ou l'incertitude : les cas à risque élevé doivent être abordés tôt afin de lever les incertitudes majeures au plus vite. La *priorité fonctionnelle* (haute, moyenne, basse) reflète l'importance du cas pour la valeur d'usage : les cas les plus attendus sont livrés en premier. Le nombre d'itérations estimé en découle.

Cas d'utilisation	Risque	Priorité	Nb d'itérations
Jouer une partie	Haut	Haute	5
Rejoindre un salon	Moyen	Haute	3
Créer / configurer un salon	Moyen	Haute	3
Gérer son salon (exclure, fermer)	Bas	Moyenne	2
Noter une question (up/down)	Bas	Moyenne	2
S'authentifier	Moyen	Moyenne	2
Récupérer son mot de passe	Bas	Basse	1
Consulter son profil et ses statistiques	Bas	Moyenne	2
Soumettre une question	Moyen	Basse	3
Modérer les questions soumises	Moyen	Basse	2
Souscrire un abonnement	Haut	Basse	3

Le cas *jouer une partie* cumule risque et priorité élevés : c'est le cœur du système, et la synchronisation temps réel en constitue le principal défi technique ; il est donc traité en priorité et sur le plus grand nombre d'itérations. À l'inverse, le cas *souscrire*

un abonnement présente un risque élevé (intégration d'un paiement) mais une priorité basse : la valeur de jeu peut être livrée sans lui, il est donc planifié tardivement, tout en étant prototypé tôt pour lever le risque d'intégration. Cet ordonnancement se traduit concrètement par le découpage en incréments présenté ci-dessous.

5.5 Découpage en incréments

La construction de l'application est planifiée en incréments successifs. Chaque incrément livre un ensemble cohérent et utilisable, et s'appuie sur le précédent. L'ordre suit les priorités établies ci-dessus : on bâtit d'abord le cœur jouable, puis on enrichit progressivement.

Un **incrément préliminaire** (incrément 0) constitue le socle sur lequel tout repose. Il comporte deux volets : d'une part un prototype technique isolé de la synchronisation temps réel — le principal risque du projet, levé ainsi dès le départ ; d'autre part la mise en place du modèle de données (question, réponse, tags, pack) et le chargement d'un premier ensemble de packs officiels, indispensable pour que le jeu dispose de questions à présenter. Cet incrément ne livre pas de fonctionnalité visible par l'utilisateur final mais conditionne tous les suivants.

1. **Noyau de jeu.** Moteur de la partie : les joueurs rejoignent une session (au moyen d'un mécanisme minimal à ce stade, par exemple un code de salon), répondent aux questions tirées de la base de données, et un score est calculé puis affiché en fin de partie en fonction des résultats de chacun. La cotation au temps est mise en place ici. C'est l'incrément fondateur, qui valide la mécanique de jeu et la synchronisation. (*Cas : jouer une partie.*)
2. **Interface de salon.** Mise en place de l'interface complète permettant de créer un salon, de lister et de rejoindre les salons publics, et de configurer la partie : choix du pack ou de la catégorie de questions, mode de jeu, type de cotation (au temps ou par validation en fin de partie) et caractère public ou privé du salon. Le second mode de cotation est introduit à ce stade, ainsi que les outils de gestion du salon par l'hôte (exclure un joueur, fermer le salon). (*Cas : rejoindre un salon, créer un salon, configurer la partie, gérer son salon.*)
3. **Vote des questions.** Ajout du vote positif/négatif à chaque question, ouvert à tous les joueurs (invités compris) afin de préserver l'accès souple, avec stockage du score en base de données et tri automatique du contenu (les questions les mieux notées sont présentées plus fréquemment). Les questions trop mal notées sont automatiquement renvoyées en file de modération. Des garde-fous contre la manipulation des votes pourront être ajoutés ultérieurement, une fois les comptes en place. (*Cas : noter une question.*)
4. **Comptes et profils.** Introduction de l'authentification, de la récupération de mot de passe, du profil personnel et des statistiques (historique de parties, taux de bonnes réponses, etc.). (*Cas : s'authentifier, récupérer son mot de passe, consulter son profil et ses statistiques.*)

5. **Modes de jeu.** Ajout de modes complémentaires à la course au score classique (élimination par système de vies, jeu en équipes, etc.), sélectionnables au moment de configurer la partie.
6. **Contenu communautaire.** Soumission de questions par les utilisateurs et mise en place de la file de modération, éventuellement assistée par intelligence artificielle. (*Cas : soumettre une question, modérer les questions soumises.*)
7. **Monétisation.** Introduction de l'abonnement (accès à l'intégralité du catalogue officiel) et des éléments cosmétiques. (*Cas : souscrire un abonnement.*)

6 Scénarios des cas d'utilisation

Ce chapitre détaille six cas d'utilisation représentatifs sous forme de fiches. Chaque fiche comporte un sommaire d'identification, les préconditions, le scénario nominal (déroulement standard), les scénarios alternatifs (variantes valides), les scénarios d'erreur (situations d'échec) et les postconditions. Ces descriptions servent de fondement aux diagrammes de séquence présentés ultérieurement.

6.1 S'authentifier

Rubrique	Description
But	Permettre à un utilisateur possédant un compte de s'identifier afin d'accéder aux fonctionnalités persistantes.
Acteur principal	Visiteur (devenant Utilisateur authentifié).
Acteurs secondaires	Aucun.
Déclencheur	L'utilisateur choisit de se connecter.
Type	Primaire, important.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.0
Responsable	van der Veen Georgé

Préconditions. L'utilisateur dispose d'un compte et n'est pas déjà connecté.

Scénario nominal.

1. L'utilisateur demande à se connecter.
2. Le système affiche le formulaire d'authentification.
3. L'utilisateur saisit son identifiant (ou son adresse e-mail) et son mot de passe.
4. Le système vérifie les informations fournies.
5. Le système ouvre la session et affiche l'espace de l'utilisateur authentifié.

Scénarios alternatifs.

- *A1 — session invité en cours* : si l'utilisateur jouait déjà en tant qu'invité, le système rattache cette session au compte lors de la connexion.

Scénarios d'erreur.

- *E1 — identifiants incorrects* : le système signale l'échec et invite à réessayer ; après plusieurs tentatives infructueuses, il propose la récupération du mot de passe.

- *E2 — compte inexistant* : le système propose la création d'un compte.
- *E3 — service d'authentification indisponible* : le système affiche un message et invite à réessayer ultérieurement.

Postconditions. L'utilisateur est authentifié et sa session est ouverte.

6.2 Rejoindre un salon

Rubrique	Description
But	Permettre à un joueur de rejoindre une partie existante, publique ou privée.
Acteur principal	Visiteur.
Acteurs secondaires	Aucun.
Déclencheur	Le joueur choisit de rejoindre un salon.
Type	Primaire, essentiel.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.0
Responsable	van der Veen Georgé

Préconditions. Au moins un salon ouvert existe ; pour un salon privé, le joueur dispose du code ou du lien d'accès.

Scénario nominal.

1. Le joueur demande à rejoindre un salon, en le sélectionnant dans la liste des salons publics.
2. Le système vérifie que le salon existe et est ouvert.
3. Le système vérifie que le salon n'a pas atteint sa capacité maximale.
4. Le système ajoute le joueur au salon.
5. Le système affiche l'état du salon (joueurs présents, configuration, attente du lancement).

Scénarios alternatifs.

- *A1 — salon privé* : le joueur saisit un code d'accès ; le système valide ce code avant de l'ajouter au salon.
- *A2 — partie déjà en cours* : selon la configuration, le joueur est placé en spectateur jusqu'à la question suivante.

Scénarios d'erreur.

- *E1 — salon plein* : le système refuse l'accès et propose d'autres salons.
- *E2 — salon inexistant ou fermé* : le système affiche un message et renvoie à la liste des salons.
- *E3 — code d'accès invalide* : le système signale l'erreur et invite à ressaisir le code.
- *E4 — joueur précédemment exclu* : si l'hôte l'avait exclu, l'accès lui est refusé.

Postconditions. Le joueur fait partie du salon et attend le lancement de la partie (ou y participe comme spectateur).

6.3 Créer un salon

Rubrique	Description
But	Permettre à un joueur de créer un salon et d'en définir les paramètres de jeu.
Acteur principal	Visiteur (devenant hôte du salon).
Acteurs secondaires	Aucun.
Déclencheur	Le joueur choisit de créer un salon.
Relation	Inclut le cas « Configurer la partie ».
Type	Primaire, essentiel.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.0
Responsable	van der Veen Georgé

Préconditions. Aucune ; le mode invité est autorisé, avec les limitations propres au visiteur.

Scénario nominal.

1. Le joueur demande la création d'un salon.
2. Le système crée le salon et désigne le joueur comme hôte.
3. Le système présente les options de configuration (cas inclus « Configurer la partie ») : choix du pack ou de la catégorie de questions, mode de jeu, type de cotation (au temps ou par validation en fin de partie), caractère public ou privé, condition de victoire (seuil de points ou nombre de questions).
4. Le joueur valide la configuration.
5. Le système ouvre le salon et fournit un lien ou un code de partage.

Scénarios alternatifs.

- *A1 — hôte non authentifié* : les packs officiels réservés à l'abonnement ne sont pas proposés, et la configuration n'est pas sauvegardée entre deux sessions.
- *A2 — salon privé* : le système génère un code ou un lien d'accès restreint.

Scénarios d'erreur.

- *E1 — aucun pack disponible* : le système empêche la création et invite à sélectionner un pack.
- *E2 — configuration incohérente* : le système signale les paramètres en conflit et bloque la validation.

Postconditions. Un salon configuré et ouvert existe ; l'hôte peut y inviter d'autres joueurs.

6.4 Jouer une partie

Rubrique	Description
But	Dérouler une partie de quiz dans un salon, de la première question au classement final.

Rubrique	Description
Acteur principal	Visiteur (plusieurs joueurs participent simultanément).
Acteurs secondaires	Aucun.
Déclencheur	L'hôte lance la partie.
Relation	Étendu par le cas « Noter une question ».
Type	Primaire, essentiel.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.1
Responsable	van der Veen Georgé

Préconditions. Un salon configuré existe et le nombre minimal de joueurs requis est présent.

Scénario nominal.

1. L'hôte lance la partie.
2. Le système sélectionne une question (selon le pack et le tri par votes) et l'affiche simultanément à tous les joueurs, en démarrant le décompte du temps.
3. Chaque joueur saisit sa réponse.
4. Le système évalue les réponses selon le mode de cotation au temps : le joueur le plus rapide à répondre correctement marque le plus de points.
5. Le système met à jour et affiche le classement en direct.
6. Le système répète les étapes 2 à 5 pour chaque question, jusqu'à ce que la condition de fin soit atteinte (seuil de points ou nombre de questions épuisé).
7. Le système affiche le classement final.

Scénarios alternatifs.

- *A1 — cotation par validation en fin de partie* : les réponses sont collectées puis jugées à la fin de la partie, avec une tolérance orthographique accrue.
- *A2 — noter une question* : à l'issue d'une question, un joueur attribue un vote positif ou négatif (cas en extension).
- *A3 — mode élimination* : un joueur ayant épuisé ses vies est éliminé et bascule en spectateur.

Scénarios d'erreur.

- *E1 — déconnexion d'un joueur* : le système le retire de la question en cours ; il peut se reconnecter et reprendre tant que la partie n'est pas terminée.
- *E2 — déconnexion de l'hôte* : le système transfère le rôle d'hôte à un autre joueur ou met la partie en pause.
- *E3 — temps écoulé sans réponse* : la question est comptabilisée comme non répondue (aucun point) pour le joueur concerné.
- *E4 — plus aucun joueur connecté* : la partie est interrompue et le salon fermé.

Postconditions. La partie est terminée et un classement final est établi ; les résultats des joueurs authentifiés sont enregistrés dans leurs statistiques.

6.5 Soumettre une question

Rubrique	Description
But	Permettre à un utilisateur authentifié de proposer une question au catalogue communautaire.
Acteur principal	Utilisateur authentifié.
Acteurs secondaires	Service d'intelligence artificielle (suggestion de catégories), traité comme mécanisme interne à ce stade.
Déclencheur	L'utilisateur choisit de soumettre une question.
Type	Primaire, important.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.0
Responsable	van der Veen Georgé

Préconditions. L'utilisateur est authentifié.

Scénario nominal.

1. L'utilisateur ouvre le formulaire de création de question.
2. Il saisit l'énoncé, la ou les réponses acceptées et, éventuellement, un média associé.
3. Le système propose des tags de catégorie (assistance par intelligence artificielle) ; l'utilisateur les ajuste si besoin.
4. L'utilisateur soumet la question.
5. Le système enregistre la question avec le statut « en attente de modération ».
6. Le système confirme la soumission à l'utilisateur.

Scénarios alternatifs.

- *A1 — ajout à un pack existant* : l'utilisateur rattache la question à l'un de ses packs déjà créés.
- *A2 — soumission groupée* : l'utilisateur soumet un pack entier en une fois.

Scénarios d'erreur.

- *E1 — champ obligatoire manquant* : le système refuse la soumission et indique les champs à compléter.
- *E2 — média non conforme* : le système rejette un média au format ou à la taille non autorisés.
- *E3 — doublon détecté* : le système avertit que la question existe déjà et propose de l'éditer ou d'annuler.

Postconditions. La question est enregistrée en file de modération ; elle n'est pas encore jouable.

6.6 Souscrire un abonnement

Rubrique	Description
But	Permettre à un utilisateur authentifié de souscrire un abonnement donnant accès à l'intégralité du catalogue officiel et aux éléments cosmétiques.

Rubrique	Description
Acteur principal	Utilisateur authentifié.
Acteurs secondaires	Prestataire de paiement, traité comme service externe à ce stade de l'analyse.
Déclencheur	L'utilisateur choisit de souscrire un abonnement.
Type	Primaire, secondaire.
Dates	Création : 15/03/2026 — dernière modification : 11/06/2026.
Version	1.0
Responsable	van der Veen Georgé

Préconditions. L'utilisateur est authentifié et ne dispose pas déjà d'un abonnement actif.

Scénario nominal.

1. L'utilisateur sélectionne l'offre d'abonnement.
2. Le système présente le récapitulatif (contenu, prix, conditions).
3. L'utilisateur confirme et fournit ses informations de paiement.
4. Le système transmet la demande au prestataire de paiement.
5. Le prestataire confirme le paiement.
6. Le système active l'abonnement et débloquent le catalogue officiel complet ainsi que les cosmétiques.
7. Le système confirme l'activation à l'utilisateur.

Scénarios alternatifs.

- *A1 — code promotionnel* : l'utilisateur saisit un code ; le système ajuste le montant en conséquence.

Scénarios d'erreur.

- *E1 — paiement refusé* : le système signale l'échec ; l'abonnement n'est pas activé et l'utilisateur peut réessayer.
- *E2 — prestataire indisponible* : le système diffère l'opération et invite à réessayer ultérieurement.
- *E3 — abonnement déjà actif* : le système informe l'utilisateur et n'effectue aucun nouveau paiement.

Postconditions. L'abonnement est actif et l'utilisateur accède au contenu réservé.

7 Diagrammes de séquence système (boîte noire)

Cette section présente les diagrammes de séquence au niveau *système* (boîte noire) : le système est vu comme un participant unique, et seuls les échanges entre les acteurs et le système sont représentés. Chaque cas est illustré par son scénario nominal, suivi d'un scénario alternatif ou d'erreur représentatif tiré de sa fiche (chapitre précédent). Conformément à la notation UML 1.4.1, ces variantes sont représentées par des diagrammes séparés plutôt que par des fragments combinés. Par convention, l'acteur y est désigné par son rôle dans le cas considéré — « Joueur » ou « Utilisateur » — en correspondance avec

les acteurs Visiteur et Utilisateur authentifié des fiches. Le raffinement en diagrammes de conception (boîte blanche), faisant apparaître les objets internes, est présenté ultérieurement.

7.1 S'authentifier

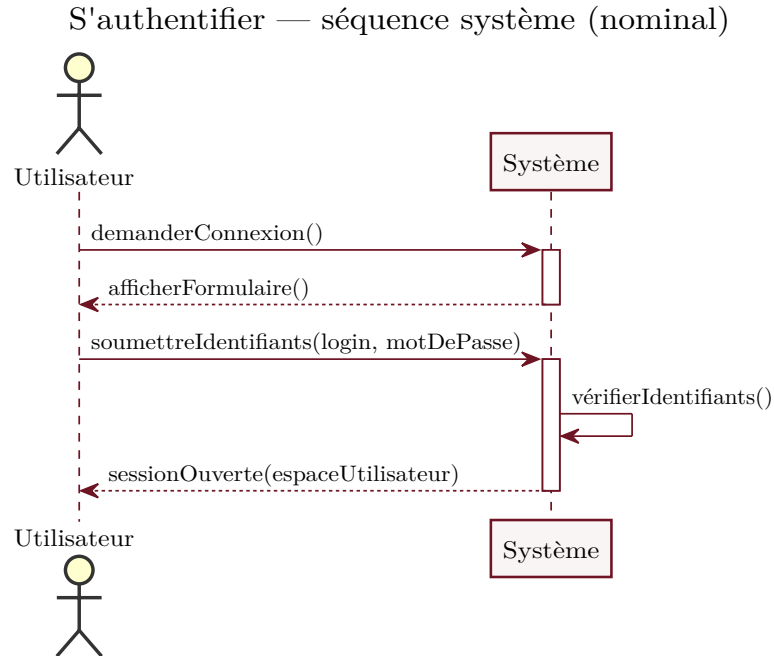


Fig. 2. — Séquence système — « S'authentifier » (scénario nominal).

S'authentifier — séquence système (E1 : identifiants incorrects)

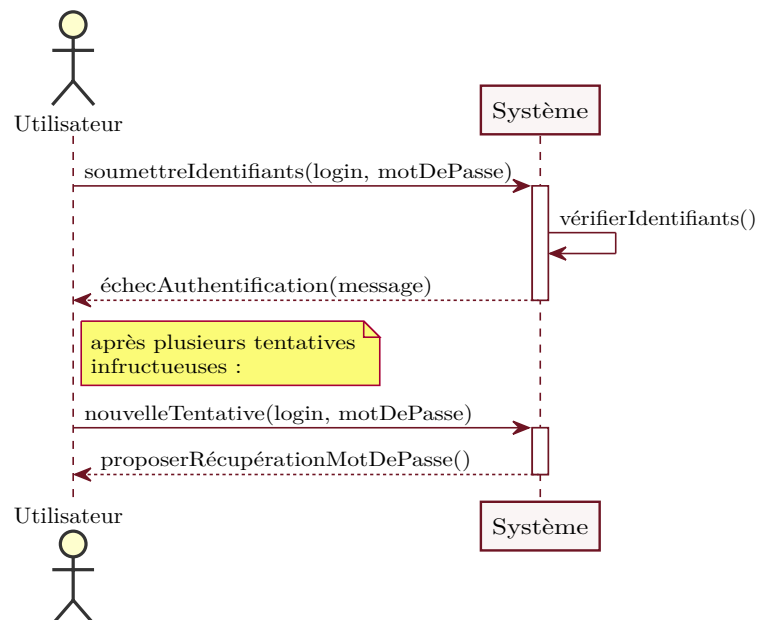


Fig. 3. — Séquence système — « S'authentifier » (scénario d'erreur E1 : identifiants incorrects).

7.2 Rejoindre un salon

Rejoindre un salon — séquence système (nominal)

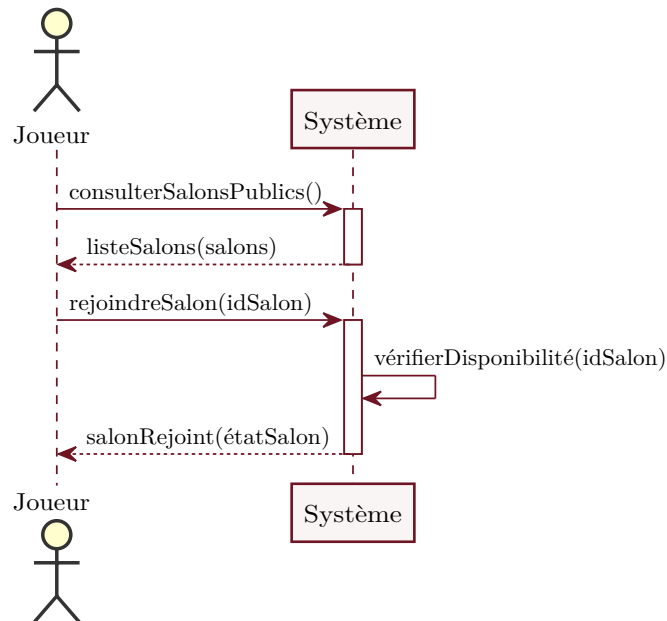


Fig. 4. – Séquence système — « Rejoindre un salon » (scénario nominal).

Rejoindre un salon — séquence système (A1 : salon privé)

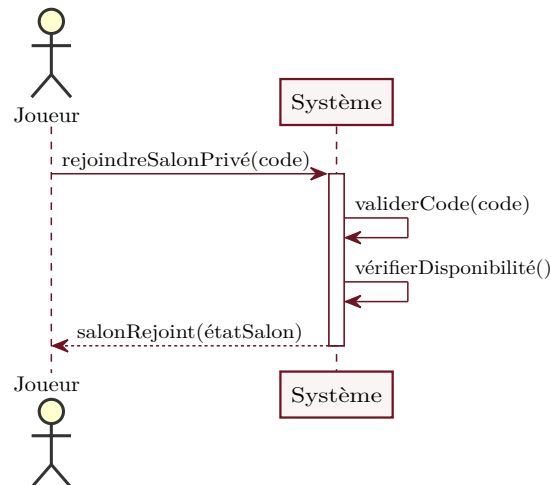


Fig. 5. – Séquence système — « Rejoindre un salon » (scénario alternatif A1 : salon privé).

7.3 Créer un salon

Créer un salon — séquence système (nominal)

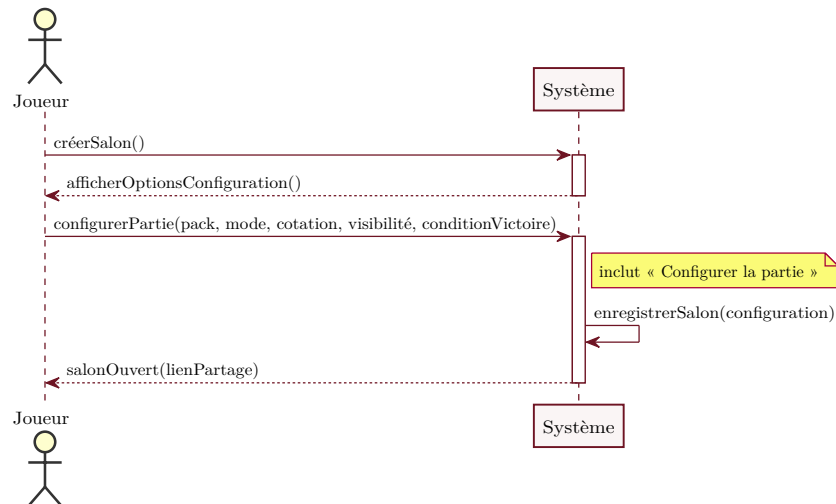


Fig. 6. – Séquence système — « Créer un salon » (scénario nominal).

Créer un salon — séquence système (E2 : configuration incohérente)

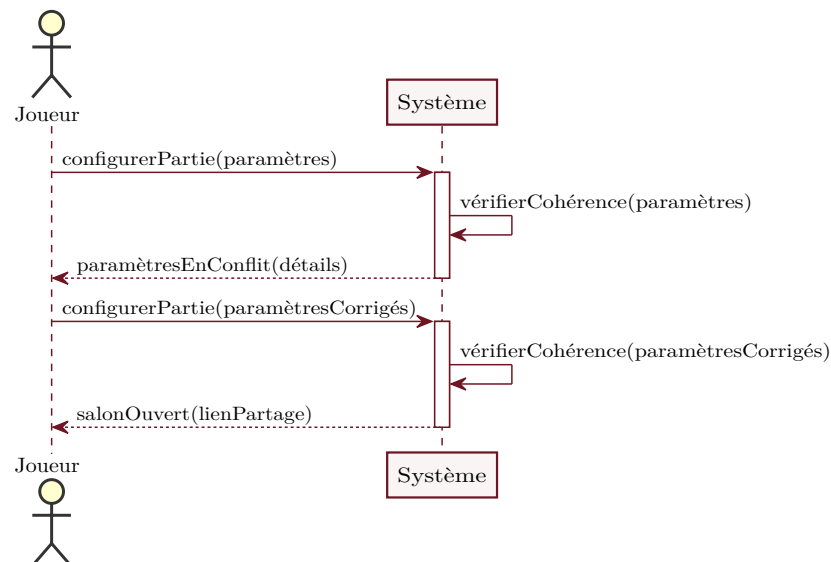


Fig. 7. – Séquence système — « Créer un salon » (scénario d'erreur E2 : configuration incohérente).

7.4 Jouer une partie

Jouer une partie — séquence système (nominal)

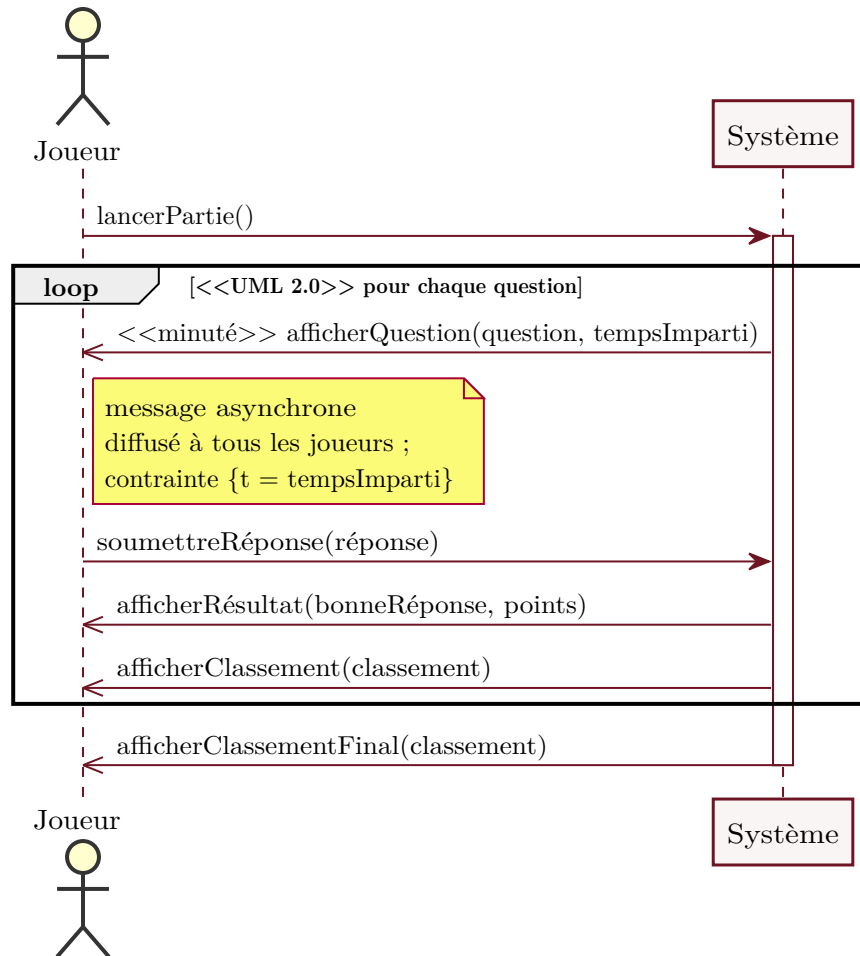


Fig. 8. – Séquence système — « Jouer une partie » (scénario nominal).

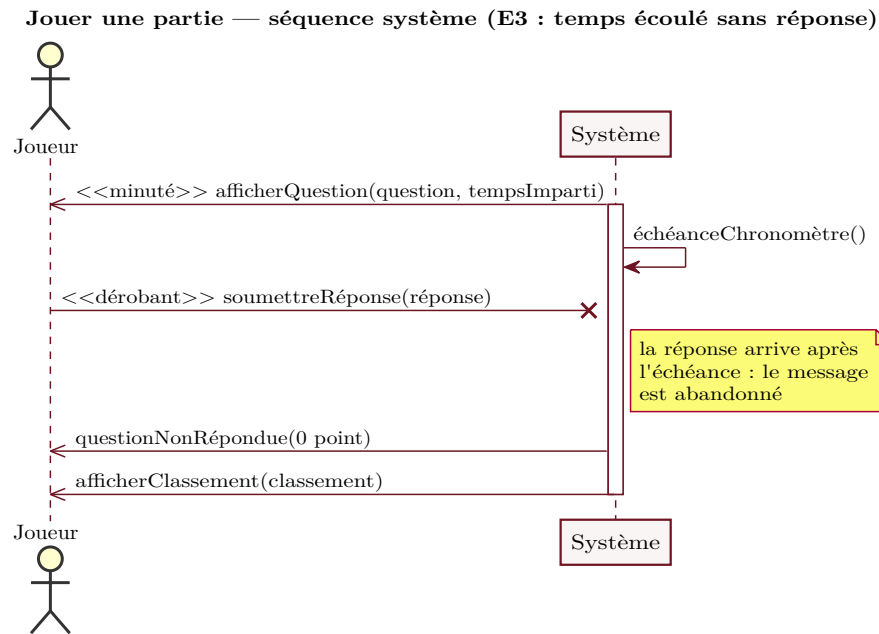


Fig. 9. – Séquence système — « Jouer une partie » (scénario d’erreur E3 : temps écoulé sans réponse).

7.5 Soumettre une question

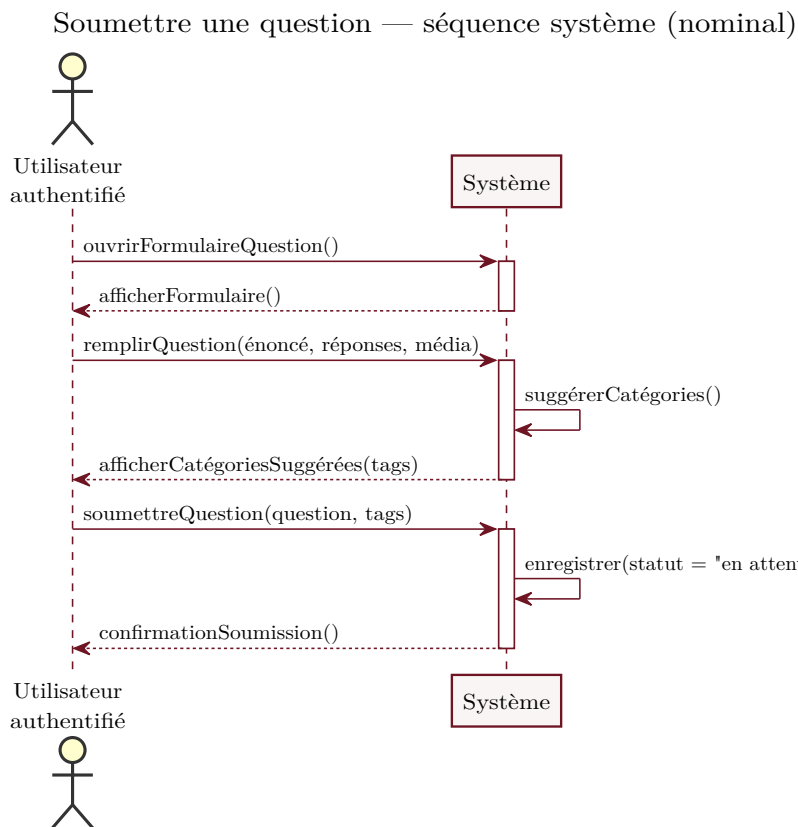


Fig. 10. – Séquence système — « Soumettre une question » (scénario nominal).

Soumettre une question — séquence système (E1 : champ obligatoire manquant)

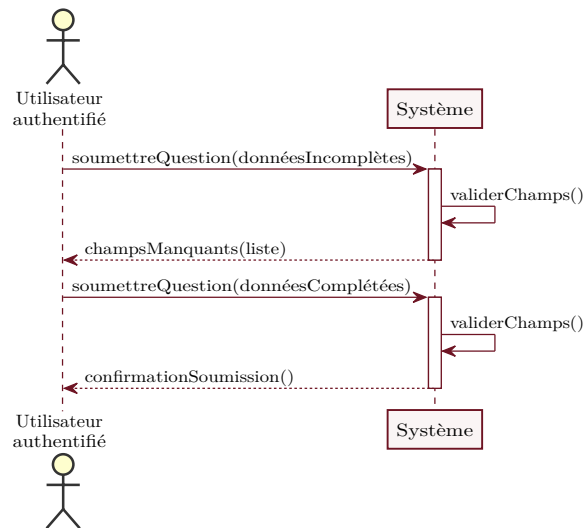


Fig. 11. – Séquence système — « Soumettre une question » (scénario d’erreur E1 : champ obligatoire manquant).

7.6 Souscrire un abonnement

Souscrire un abonnement — séquence système (nominal)

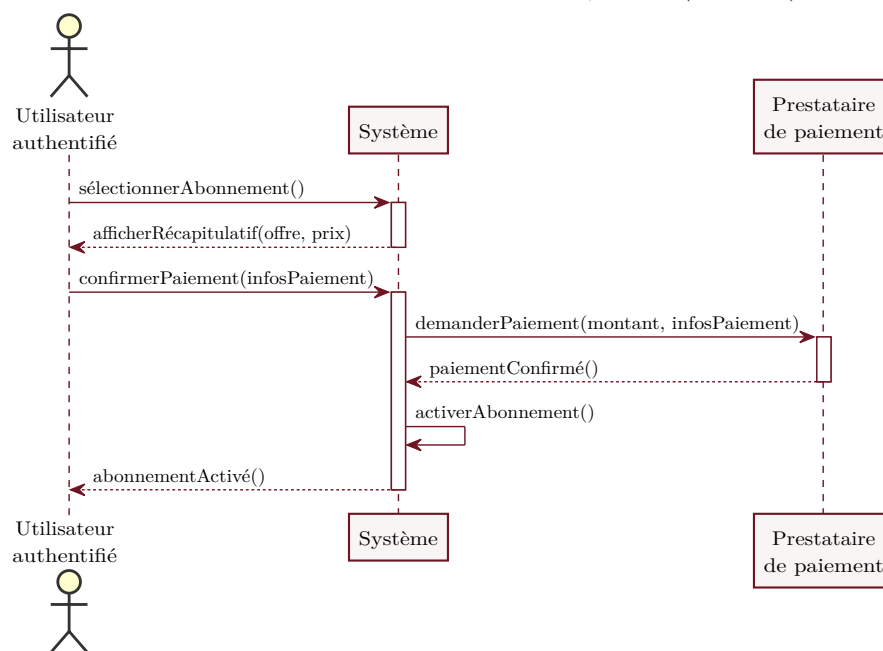


Fig. 12. – Séquence système — « Souscrire un abonnement » (scénario nominal).

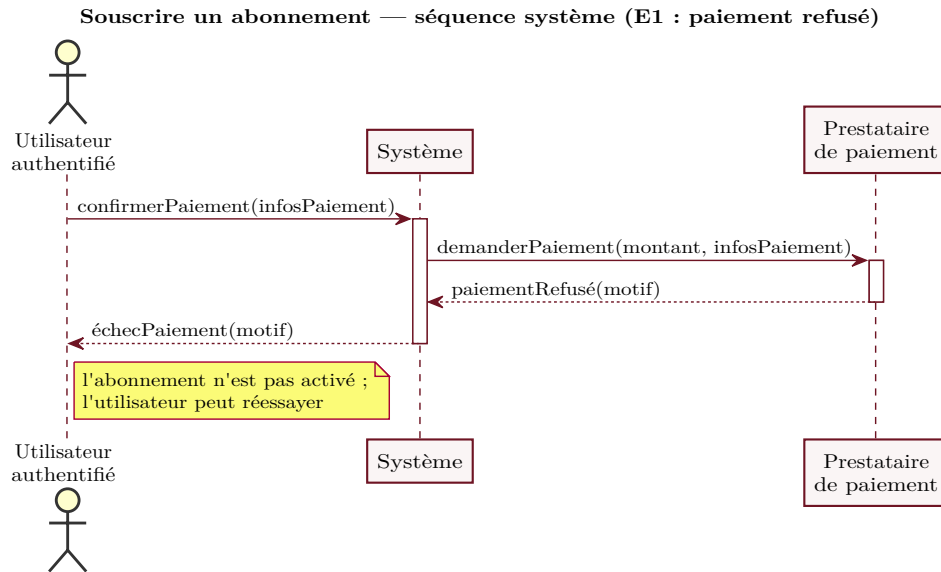


Fig. 13. — Séquence système — « Souscrire un abonnement » (scénario d'erreur E1 : paiement refusé).

8 Diagrammes de séquence de conception

Cette section raffine les diagrammes de séquence système en ouvrant la « boîte noire » : le système est décomposé en objets selon le découpage de Jacobson — objets d'interface (*boundary*), objets de contrôle (*control*) et objets entités (*entity*) — l'accès aux données étant assuré par une base de données unique. Apparaissent ici des messages absents au niveau système : la création et la destruction d'objets (stéréotypes « create » et « destroy »), ainsi que les échanges avec la persistance. Comme au niveau système, chaque scénario nominal est suivi d'un scénario alternatif ou d'erreur ; le message *dérobant* — qui n'aboutit que si le récepteur est en mesure de le traiter, et est abandonné sinon — y apparaît dans le scénario du temps écoulé.

8.1 Jouer une partie

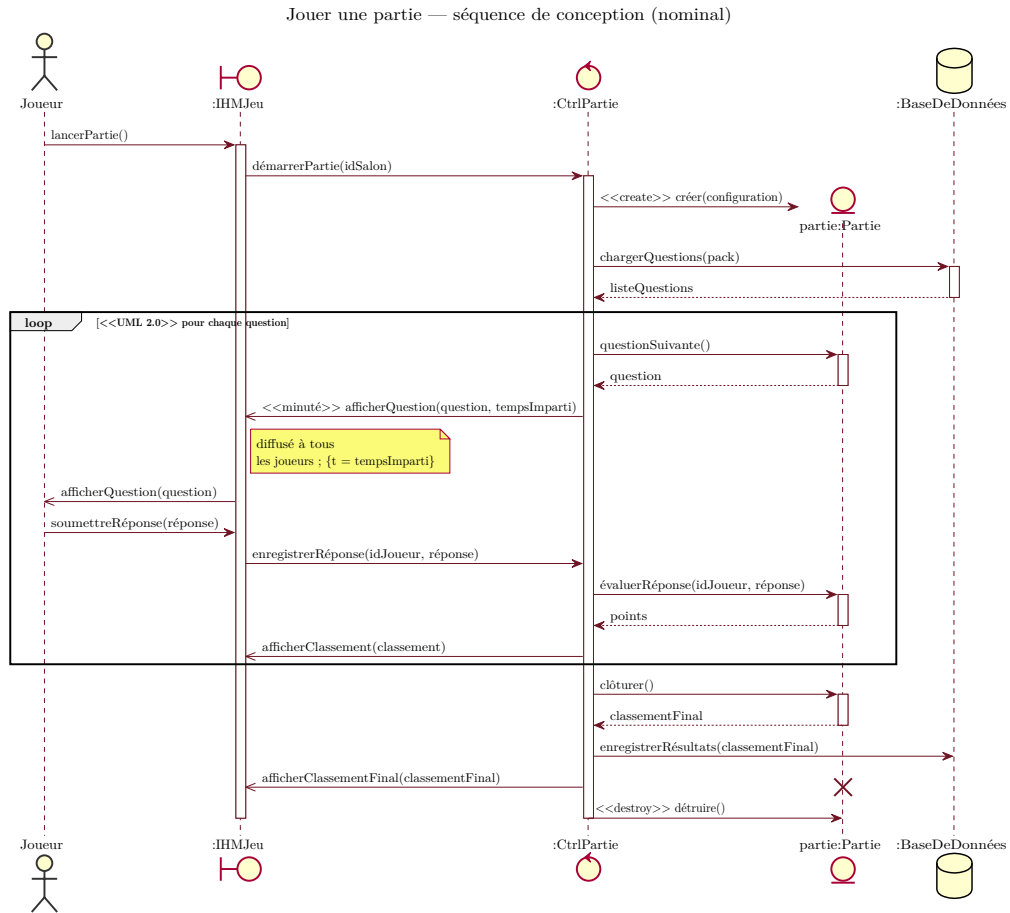


Fig. 14. – Séquence de conception — « Jouer une partie » (scénario nominal).

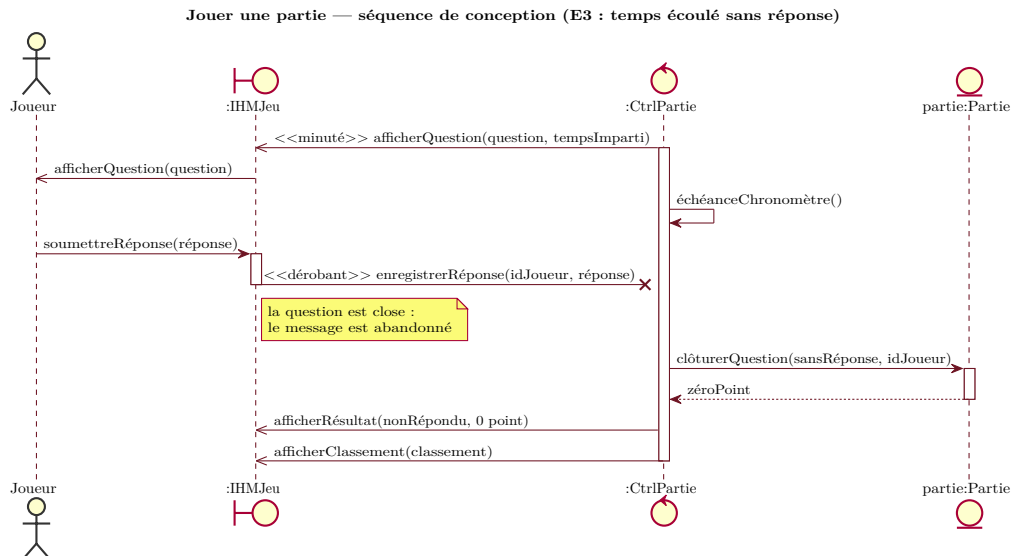


Fig. 15. – Séquence de conception — « Jouer une partie » (E3 : temps écoulé, message dérobant).

8.2 S'authentifier

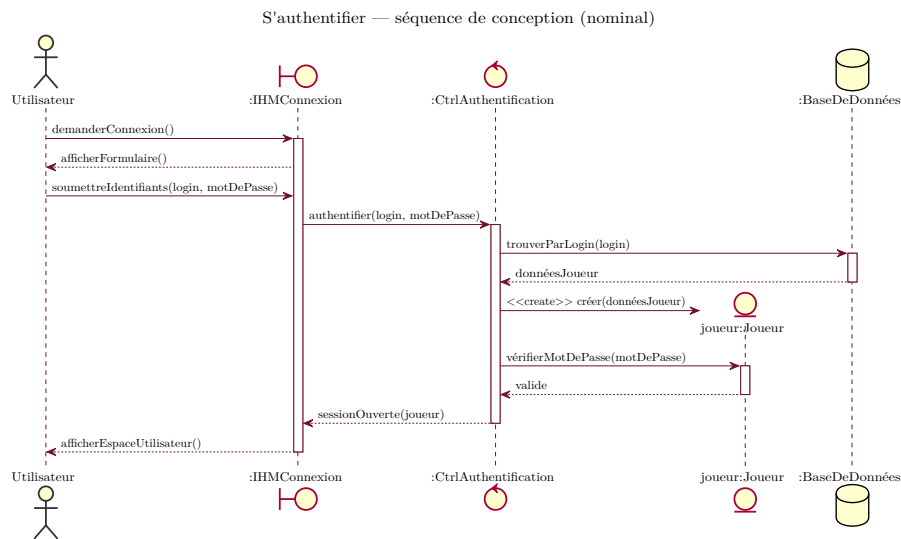


Fig. 16. — Séquence de conception — « S'authentifier » (scénario nominal).

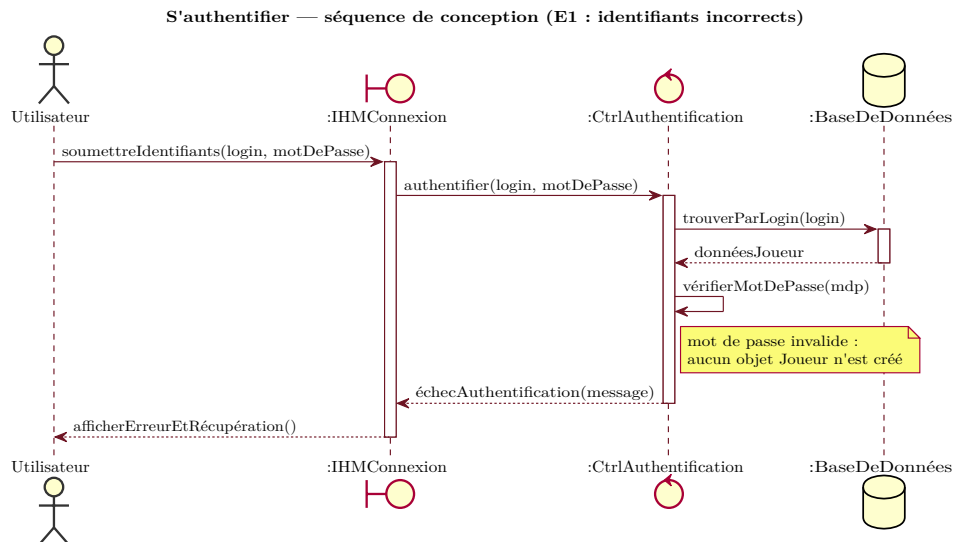


Fig. 17. — Séquence de conception — « S'authentifier » (E1 : identifiants incorrects).

8.3 Rejoindre un salon

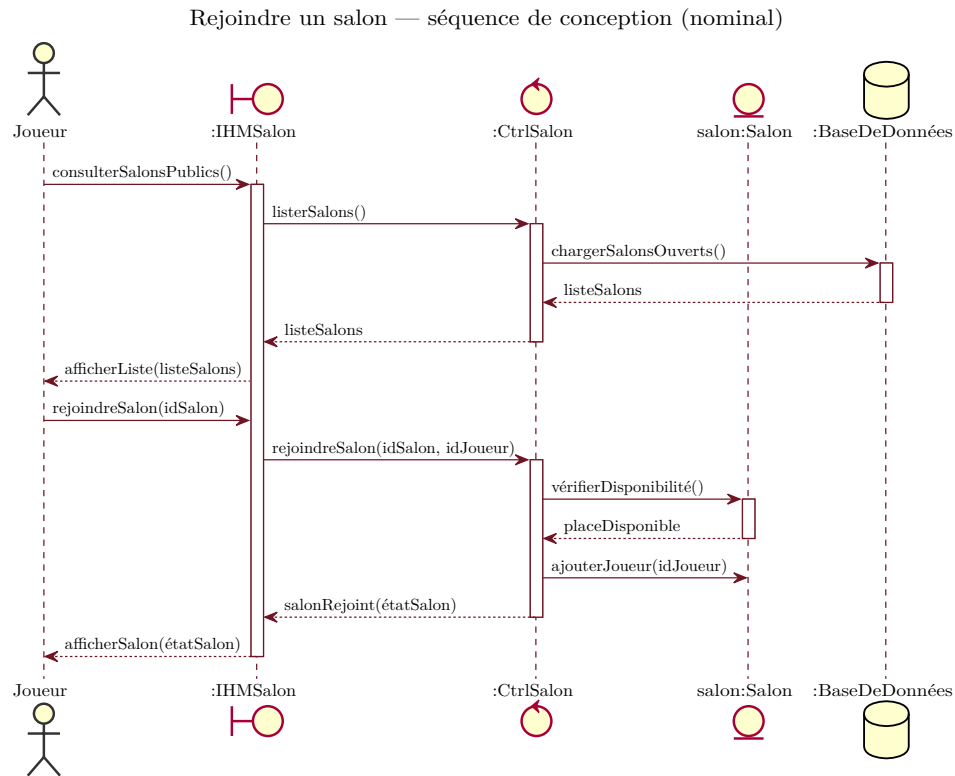


Fig. 18. – Séquence de conception — « Rejoindre un salon » (scénario nominal).

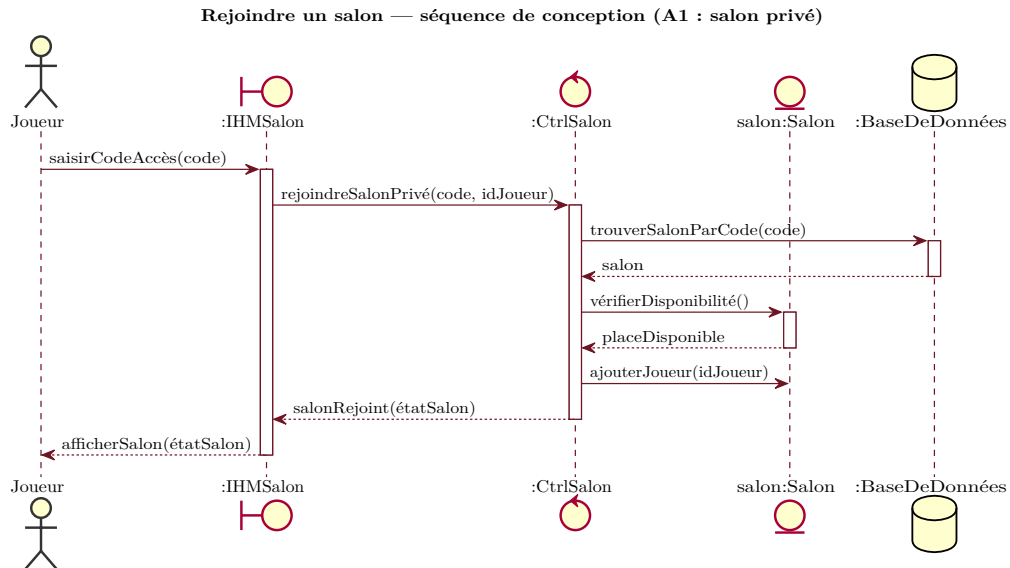


Fig. 19. – Séquence de conception — « Rejoindre un salon » (A1 : salon privé).

8.4 Créer un salon

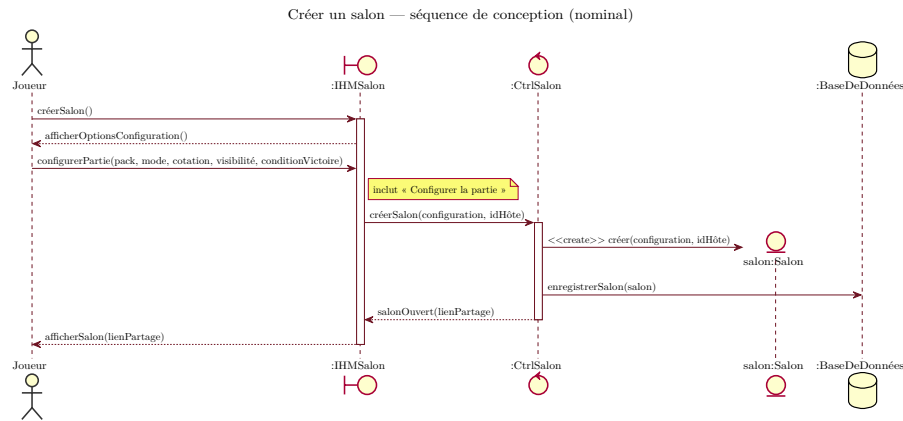


Fig. 20. – Séquence de conception — « Créer un salon » (scénario nominal).

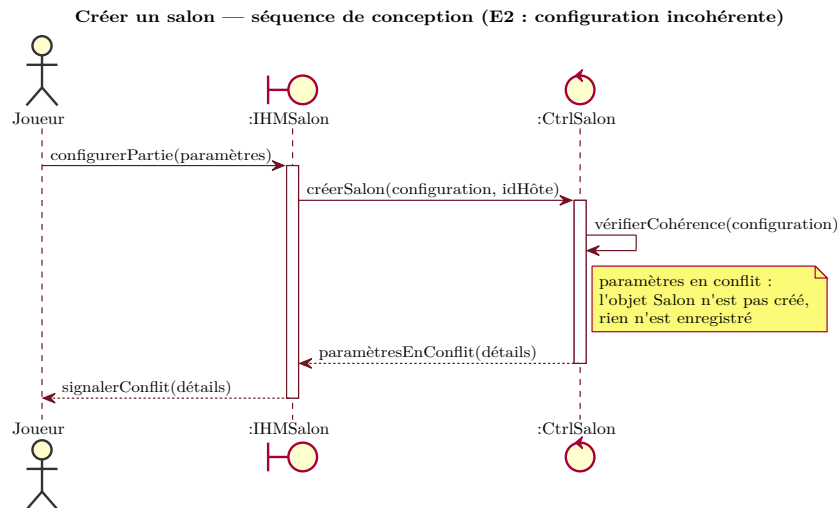


Fig. 21. – Séquence de conception — « Créer un salon » (E2 : configuration incohérente).

8.5 Soumettre une question

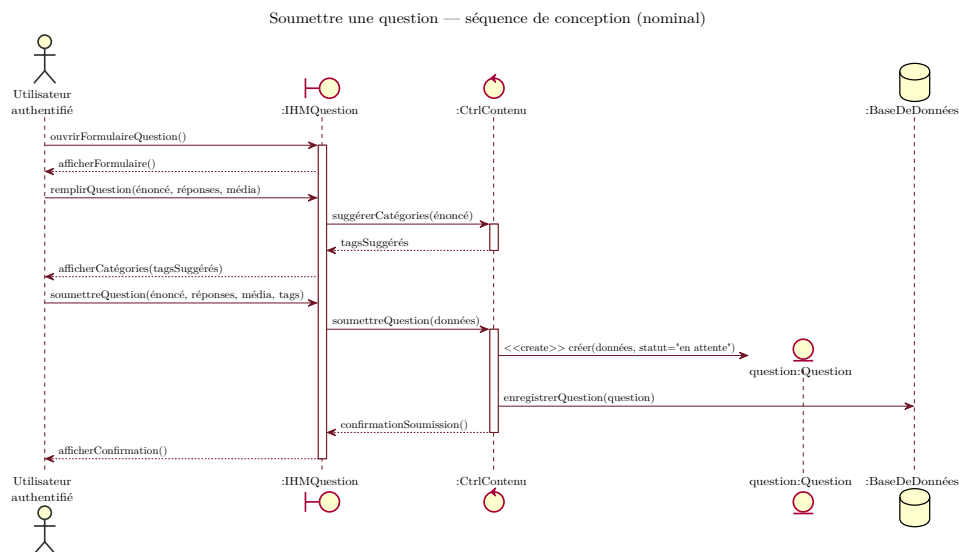


Fig. 22. – Séquence de conception — « Soumettre une question » (scénario nominal).

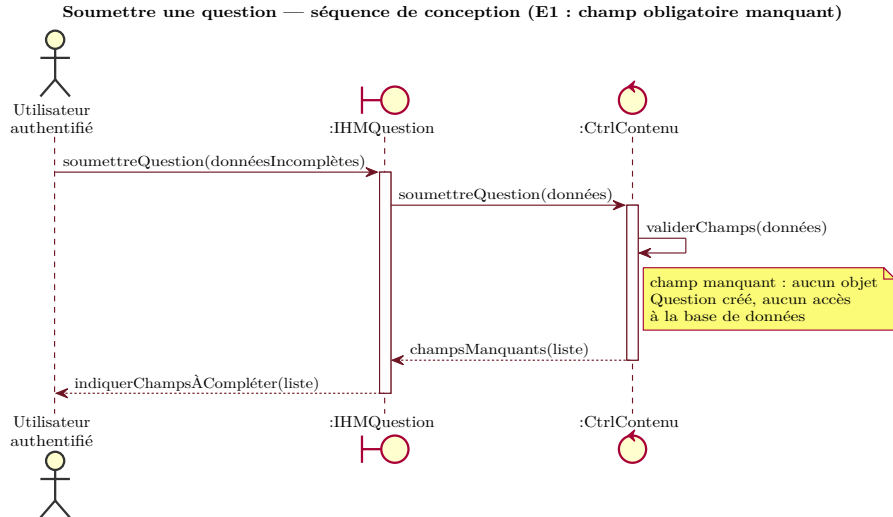


Fig. 23. – Séquence de conception — « Soumettre une question » (E1 : champ obligatoire manquant).

8.6 Souscrire un abonnement

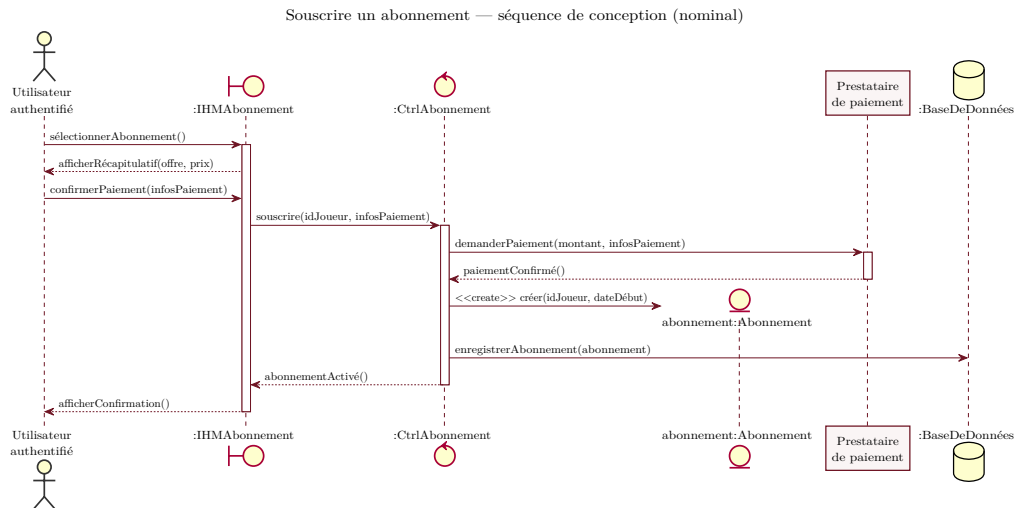


Fig. 24. – Séquence de conception — « Souscrire un abonnement » (scénario nominal).

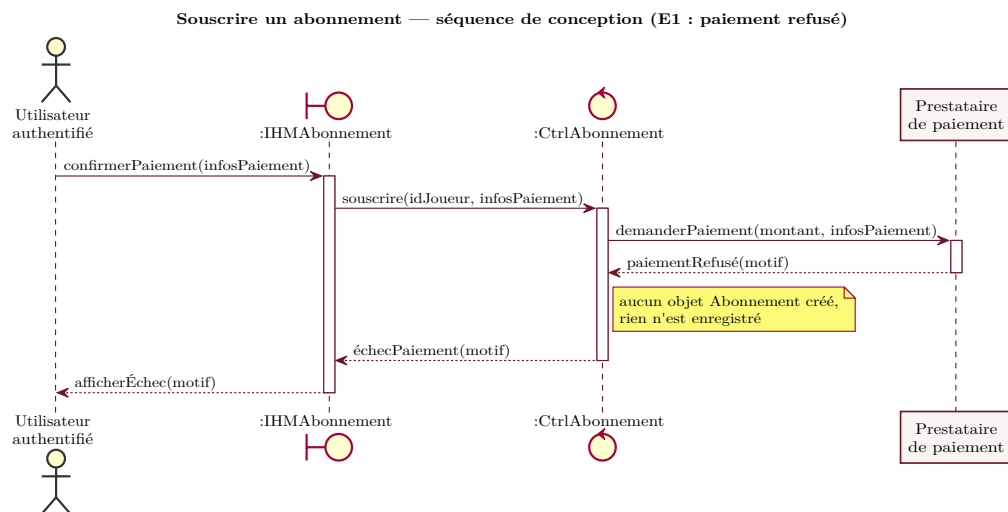


Fig. 25. – Séquence de conception — « Souscrire un abonnement » (E1 : paiement refusé).

ils sont intégrés à **Salon**. De même, les statistiques d'un membre, en relation un-à-un obligatoire avec celui-ci, sont intégrées directement à la classe **Membre**, une relation 1-1 systématique justifiant rarement une classe séparée.

Classes d'association et associations plusieurs-à-plusieurs. Lorsqu'une association plusieurs-à-plusieurs porte des données propres au lien, elle est modélisée par une classe d'association : **Participation** (score, rang, vies) sur le lien **Joueur-Partie**, et **Vote** (valeur) sur le lien **Joueur-Question**. En revanche, l'association **Question-Tag** ne porte aucune donnée : elle reste une association plusieurs-à-plusieurs simple. La table de jointure correspondante n'apparaîtra qu'au niveau de l'implémentation relationnelle, et non dans le modèle conceptuel.

Expression des contraintes. Les règles non exprimables par la seule notation graphique sont indiquées entre accolades {...}, conformément à la notation des contraintes d'UML 1.4. Le langage OCL, normalisé plus tardivement avec UML 2.0, n'est pas employé tel quel afin de rester cohérent avec la version de référence.

10 Diagrammes de collaboration

Les diagrammes de collaboration présentent la même information que les diagrammes de séquence, organisée spatialement autour des objets : chaque message est numéroté selon son ordre chronologique. Par souci de lisibilité, chaque message est porté par sa propre flèche numérotée, plutôt que par un lien unique le long duquel les messages seraient empilés ; cette variante de présentation conserve la sémantique du diagramme de collaboration (objets, liens, numérotation séquentielle). Sont présentés, pour chaque cas, le niveau système (boîte noire) puis le niveau de conception (boîte blanche), chaque scénario nominal étant suivi du scénario alternatif ou d'erreur correspondant.

10.1 Niveau système (boîte noire)

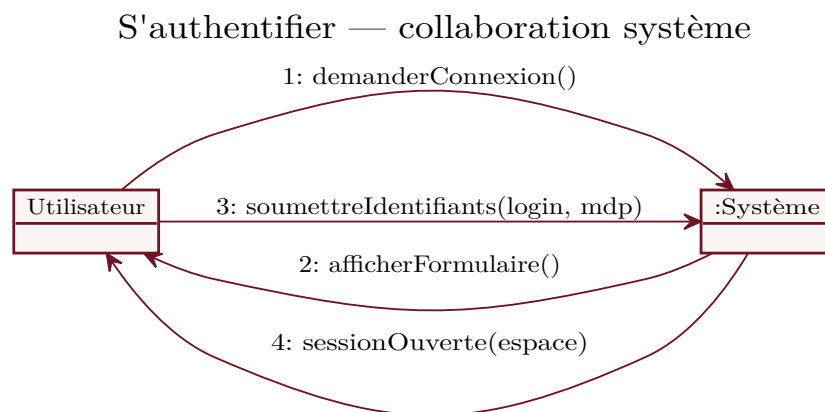


Fig. 27. – Collaboration système — « S'authentifier ».

S'authentifier — collaboration système (E1 : identifiants incorrects)

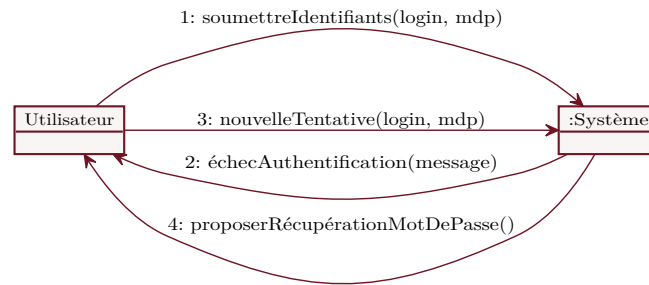


Fig. 28. — Collaboration système — « S'authentifier » (E1 : identifiants incorrects).

Rejoindre un salon — collaboration système



Fig. 29. — Collaboration système — « Rejoindre un salon ».

Rejoindre un salon — collaboration système (A1 : salon privé)

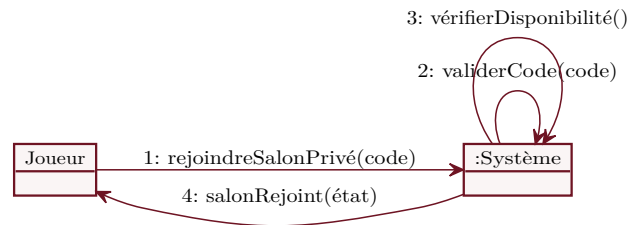


Fig. 30. — Collaboration système — « Rejoindre un salon » (A1 : salon privé).

Créer un salon — collaboration système

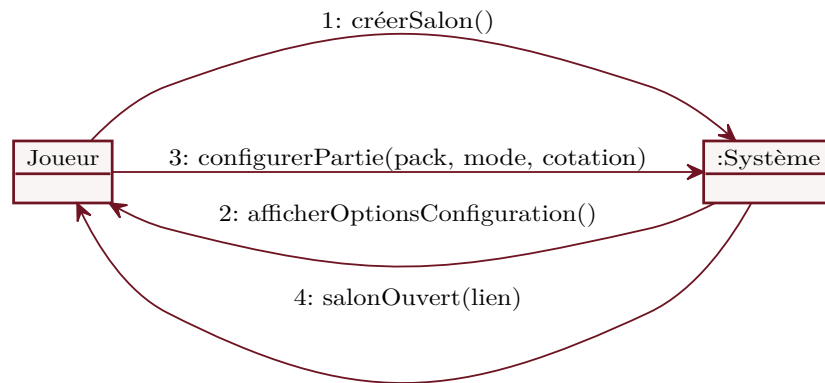


Fig. 31. – Collaboration système — « Créer un salon ».

Jouer une partie — collaboration système

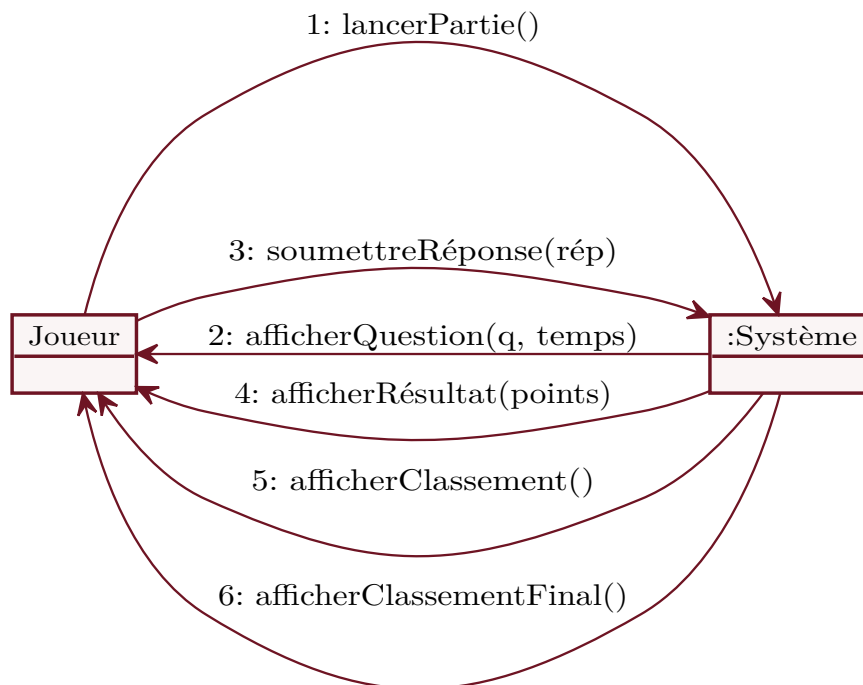


Fig. 32. – Collaboration système — « Jouer une partie ».

Jouer une partie — collaboration système (E3 : temps écoulé)

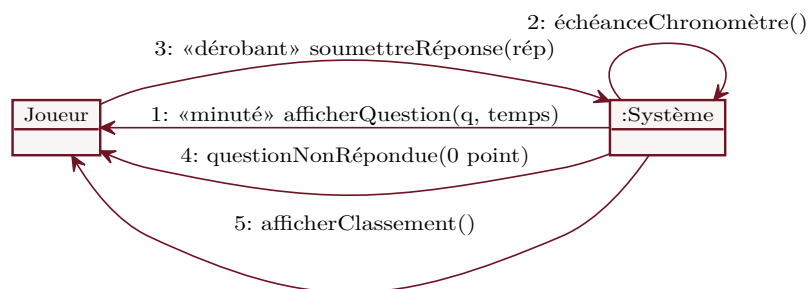


Fig. 33. – Collaboration système — « Jouer une partie » (E3 : temps écoulé).

Créer un salon — collaboration système (E2 : configuration incohérente)

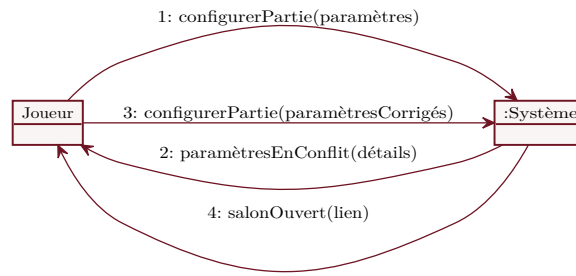


Fig. 34. – Collaboration système — « Créer un salon » (E2 : configuration incohérente).

Soumettre une question — collaboration système

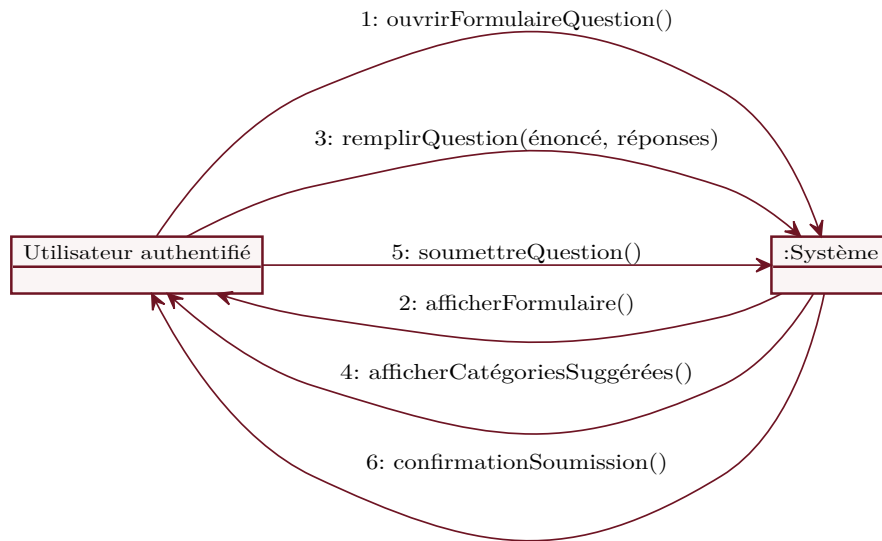


Fig. 35. – Collaboration système — « Soumettre une question ».

Soumettre une question — collaboration système (E1 : champ manquant)

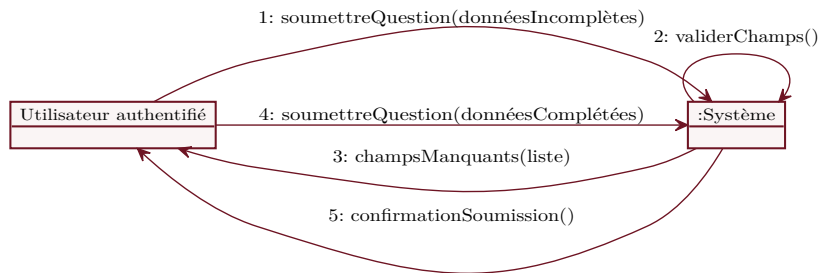


Fig. 36. – Collaboration système — « Soumettre une question » (E1 : champ manquant).

Souscrire un abonnement — collaboration système

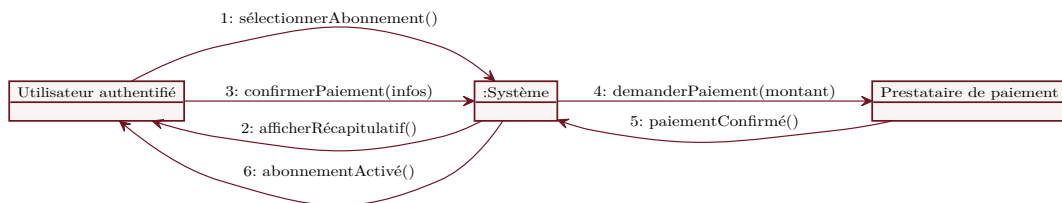


Fig. 37. – Collaboration système — « Souscrire un abonnement ».

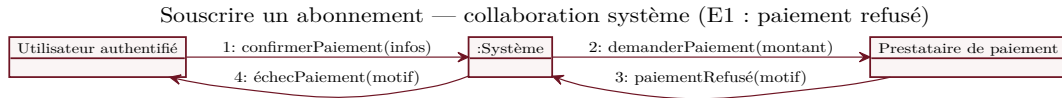


Fig. 38. – Collaboration système — « Souscrire un abonnement » (E1 : paiement refusé).

10.2 Niveau de conception (boîte blanche)

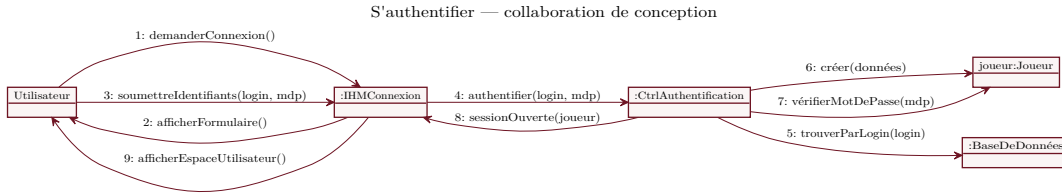


Fig. 39. – Collaboration de conception — « S'authentifier ».

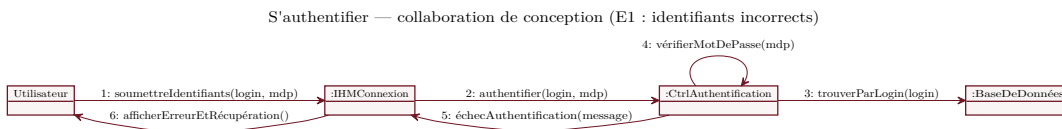


Fig. 40. – Collaboration de conception — « S'authentifier » (E1 : identifiants incorrects).



Fig. 41. – Collaboration de conception — « Rejoindre un salon ».

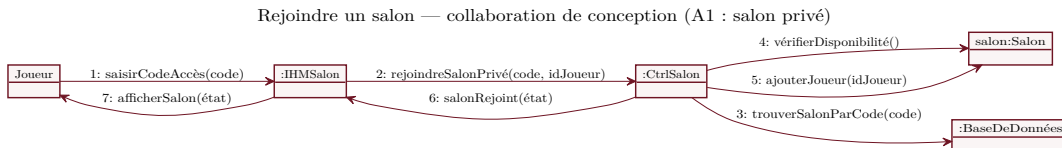


Fig. 42. – Collaboration de conception — « Rejoindre un salon » (A1 : salon privé).

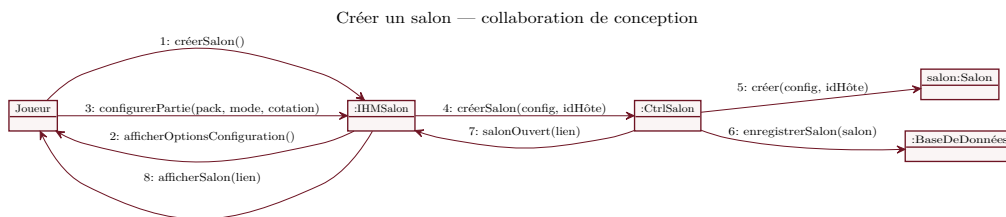


Fig. 43. – Collaboration de conception — « Créer un salon ».

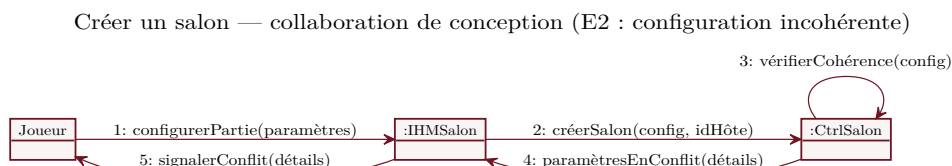


Fig. 44. – Collaboration de conception — « Créer un salon » (E2 : configuration incohérente).

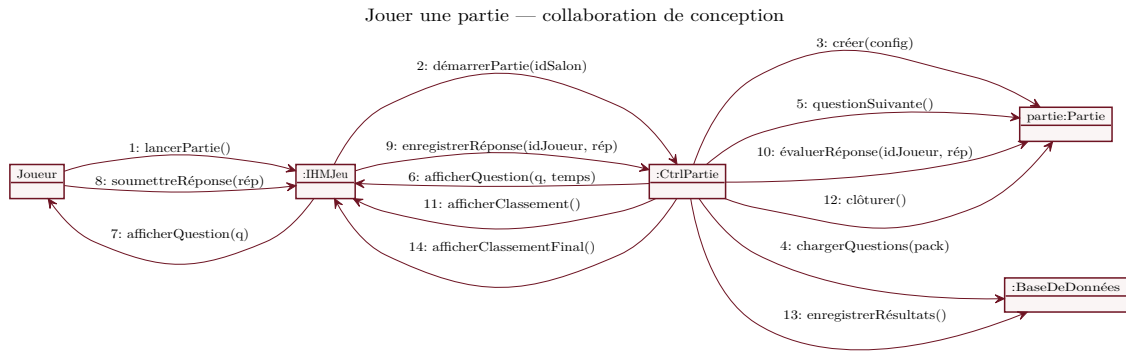


Fig. 45. – Collaboration de conception — « Jouer une partie ».

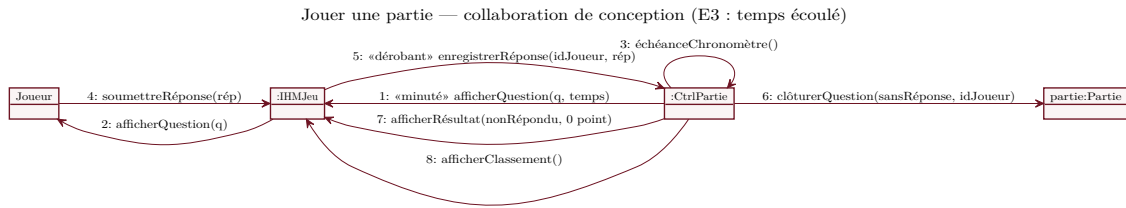


Fig. 46. – Collaboration de conception — « Jouer une partie » (E3 : temps écoulé, message dérobant).

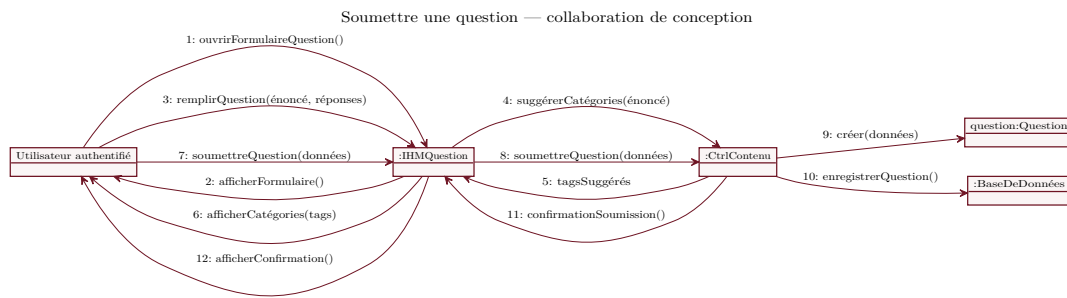


Fig. 47. – Collaboration de conception — « Soumettre une question ».

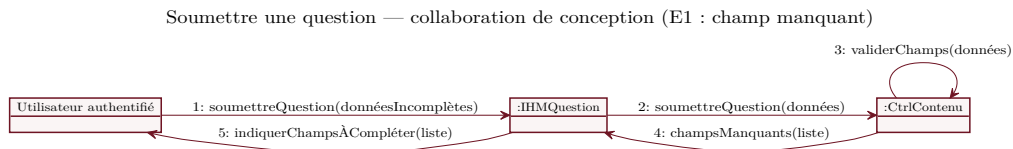


Fig. 48. – Collaboration de conception — « Soumettre une question » (E1 : champ manquant).

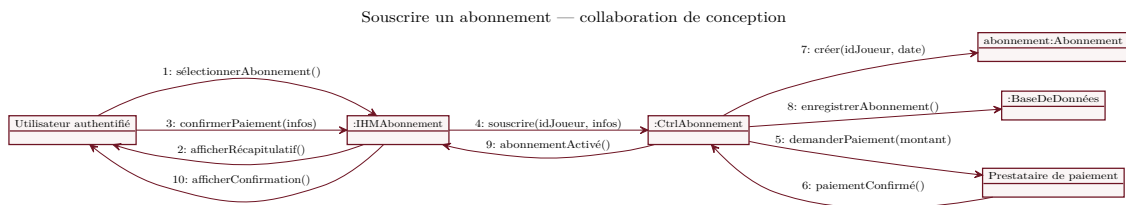


Fig. 49. – Collaboration de conception — « Souscrire un abonnement ».

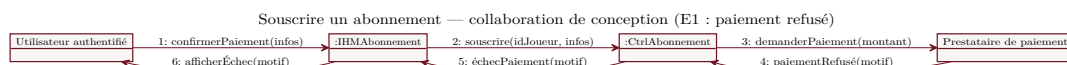


Fig. 50. – Collaboration de conception — « Souscrire un abonnement » (E1 : paiement refusé).

11 Diagramme d'activité

Le diagramme d'activité décrit la logique du déroulement d'une partie sous forme de flux d'actions, indépendamment des objets qui les réalisent. Contrairement au diagramme de séquence, centré sur les messages échangés, il met en évidence les enchaînements, les décisions et le parallélisme du processus. Les actions sont réparties en deux couloirs selon leur responsable : le joueur et le système.

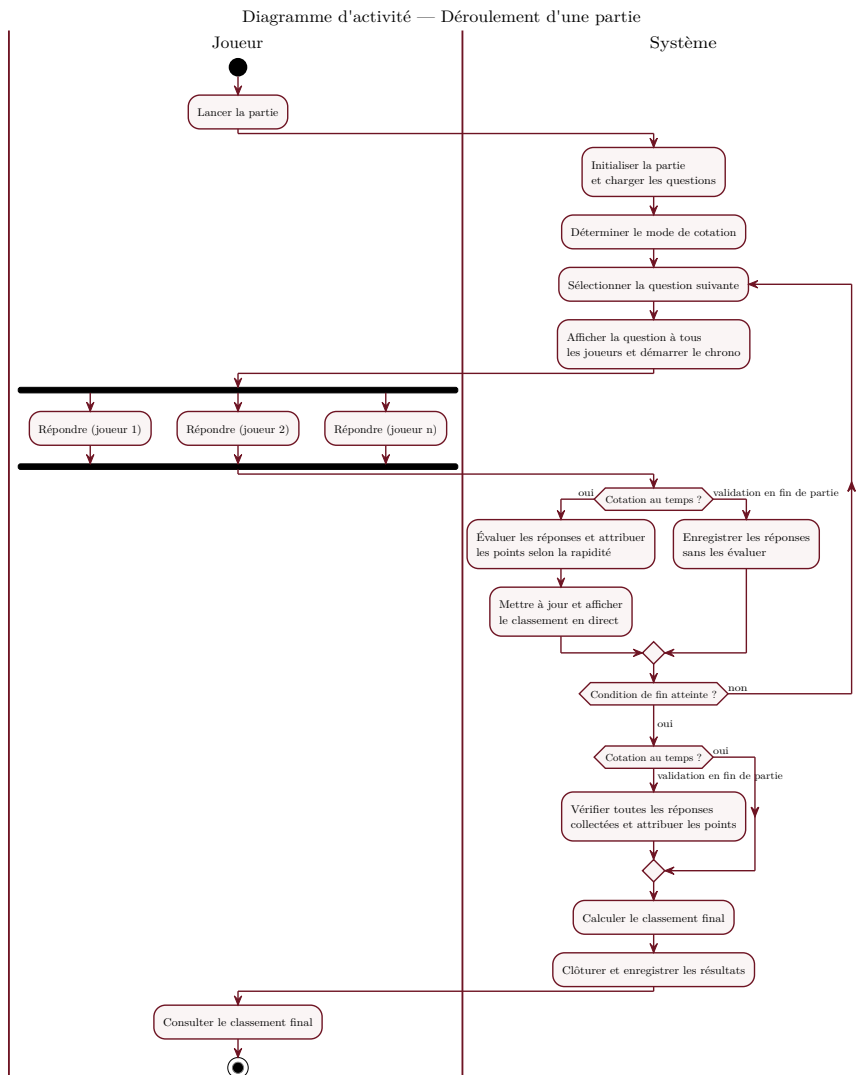


Fig. 51. – Diagramme d'activité du déroulement d'une partie (couloirs Joueur / Système).

On distingue notamment le *parallélisme* introduit par les barres de synchronisation : tous les joueurs d'un salon répondent simultanément à une même question. La condition de fin de partie structure la boucle principale, qui se répète jusqu'à ce que le seuil de points ou le nombre de questions soit atteint.

Les deux modes de cotation diffèrent par la place de l'évaluation. En cotation au temps, les réponses sont évaluées et le classement actualisé à chaque question, à l'intérieur de la boucle. En validation en fin de partie, les réponses sont seulement collectées pendant la boucle, sans évaluation ni classement intermédiaire ; elles ne sont vérifiées et créditées

qu'une fois la partie terminée, en un seul traitement. Cette distinction justifie la présence d'un second point de décision, situé après la boucle.

12 Diagramme d'états-transitions

Le diagramme d'états-transitions décrit le cycle de vie d'un objet en représentant les états par lesquels il passe et les transitions qui les relient. Le choix s'est porté sur la classe `Question`, dont le cycle de vie illustre à la fois le processus de modération et son interaction avec le système de votes.

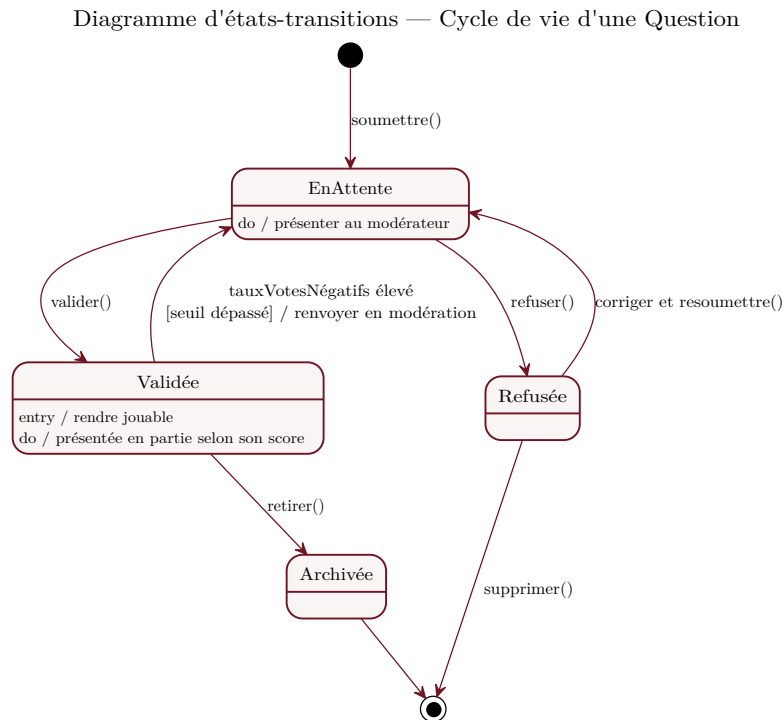


Fig. 52. – Diagramme d'états-transitions du cycle de vie d'une question.

À sa soumission, une question entre dans l'état *En attente*, où elle est présentée au modérateur. Celui-ci peut la valider — elle devient alors *Validée* et jouable, présentée en partie selon son score de votes — ou la refuser. Une question refusée peut être corrigée et resoumise. Le lien entre les votes et la modération apparaît dans la transition gardée qui ramène une question validée à l'état *En attente* lorsque son taux de votes négatifs dépasse un seuil : elle est alors automatiquement renvoyée en modération. Les activités internes aux états (*entry*, *do*) précisent les traitements associés.

13 Diagramme de packages

Le diagramme de packages organise les classes du système en paquets cohérents et met en évidence leurs dépendances. Le découpage retenu est thématique : il regroupe les entités par domaine fonctionnel, ce qui reflète les grands volets de l'application et facilite une réalisation incrémentale, chaque paquet pouvant être développé et testé de façon relativement autonome.

Diagramme de packages — Projet Cultissime

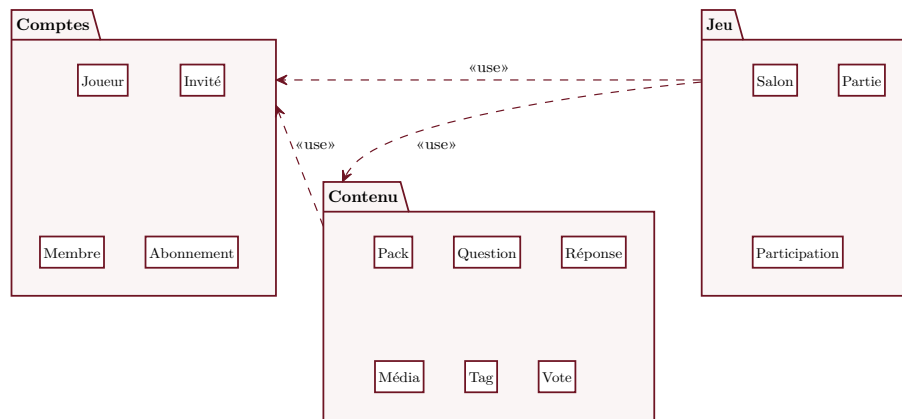


Fig. 53. – Diagramme de packages du système « Projet Cultissime ».

Trois paquets sont identifiés : *Comptes* regroupe les acteurs et leur abonnement (Joueur, Invité, Membre, Abonnement) ; *Jeu* rassemble la mécanique de partie (Salon, Partie, Participation) ; *Contenu* contient le matériel de jeu et son évaluation (Pack, Question, Réponse, Média, Tag, Vote). Les dépendances sont orientées sans cycle : le paquet *Jeu* dépend de *Comptes* et de *Contenu*, et le paquet *Contenu* dépend de *Comptes*. Il s'agit de dépendances d'accès (stéréotype « use ») : un paquet nécessite le soutien des éléments d'un autre, sans en importer le contenu.

14 Traduction des classes en code

Conformément aux consignes, les classes d'analyse des fonctionnalités majeures sont traduites en code afin de valider le passage du modèle de conception à l'implémentation. Le langage retenu pour cette traduction est C#, dans la perspective — non définitive à ce stade — d'une réalisation avec ASP.NET et Blazor (voir l'étude technologique). Ce choix reste susceptible d'évoluer ; le diagramme de classes demeure toutefois directement transposable dans un autre langage orienté objet.

L'extrait ci-dessous traduit les entités les plus représentatives : la hiérarchie d'héritage (Joueur, Invité, Membre), une énumération, l'attribut dérivé *Score* de la classe *Question*, ainsi que les deux classes d'association *Participation* et *Vote*.

```
// =====
// Traduction C# du diagramme de classes – Projet Cultissime
// (entités métier principales)
// =====

// --- Énumérations ---
public enum Visibilité { Public, Privé }
public enum ModeJeu { CourseAuScore, Élimination, Équipes }
public enum TypeCotation { AuTemps, ValidationFinManche }
public enum StatutQuestion { EnAttente, Validée, Refusée }
public enum TypePack { Officiel, Communautaire }

// --- Classe abstraite de base : Joueur ---
public abstract class Joueur
```

```
{
    protected string Pseudo { get; set; }

    public abstract void RejoindreSalon(Salon s);
    public void SoumettreRéponse(Réponse r) { /* ... */ }
}

// --- Invité : un joueur non authentifié ---
public class Invité : Joueur
{
    public override void RejoindreSalon(Salon s) { /* ... */ }
    // Contrainte {Invité : pack->isEmpty()} : un invité ne possède aucun pack.
}

// --- Membre : hérite de Joueur, porte les données persistantes ---
public class Membre : Joueur
{
    private string Login { get; set; }
    private string Email { get; set; }
    private string MotDePasse { get; set; }
    private DateTime DateInscription { get; set; }

    // Statistiques fusionnées dans Membre
    public int PartiesJouées { get; set; }
    public float TauxBonnesRéponses { get; set; }
    public float TempsRéponseMoyen { get; set; }

    public Abonnement? Abonnement { get; set; } // 0..1
    public List<Pack> PacksCréés { get; set; } = new();

    public override void RejoindreSalon(Salon s) { /* ... */ }
    public bool SAuthentifier(string login, string mdp) { /* ... */ return
true; }
    public void SoumettreQuestion(Question q) { /* ... */ }
    public Abonnement Souscrire() { /* ... */ return new Abonnement(); }
}

// --- Question : avec attribut dérivé /score ---
public class Question
{
    public string Énoncé { get; set; }
    public StatutQuestion Statut { get; set; }
    public int VotesPositifs { get; set; }
    public int VotesNégatifs { get; set; }

    // Attribut dérivé : { score = votesPositifs - votesNégatifs }
    public int Score => VotesPositifs - VotesNégatifs;

    public List<Réponse> Réponses { get; set; } = new(); // 1..*
    public List<Média> Médias { get; set; } = new(); // 0..*
    public List<Tag> Tags { get; set; } = new(); // *.*

    public void AjouterVote(int valeur) { /* ... */ }
```

```
}

// --- Participation : classe d'association Joueur-Partie ---
public class Participation
{
    public Joueur Joueur { get; set; }
    public Partie Partie { get; set; }
    public int Score { get; set; }
    public int Rang { get; set; }
    public int Vies { get; set; }
}

// --- Vote : classe d'association Joueur-Question ---
public class Vote
{
    public Joueur Joueur { get; set; }
    public Question Question { get; set; }
    public int Valeur { get; set; } // +1 ou -1
}
```

15 Glossaire

Terme	Définition
Salon	Espace de jeu en ligne réunissant plusieurs joueurs autour d'une même partie. Un salon peut être public (ouvert à tous) ou privé (accessible sur invitation).
Partie	Déroulement complet d'un jeu dans un salon, constitué d'une succession de questions, jusqu'à l'établissement d'un classement final.
Pack	Ensemble de questions regroupées, généralement par thème. Un pack peut être officiel (fourni par la plateforme) ou communautaire (créé par un utilisateur).
Question officielle	Question issue d'un pack maintenu par la plateforme, dont une partie est réservée aux abonnés.
Pack communautaire	Pack créé et soumis par un utilisateur, rendu jouable uniquement après validation par la modération.
Cotation	Règle d'attribution des points. Deux modes sont prévus : la cotation <i>au temps</i> (le plus rapide marque le plus de points) et la cotation <i>par validation en fin de partie</i> (les réponses sont jugées à la fin, ce qui assouplit la tolérance orthographique).
Vote (up/down)	Appréciation positive ou négative qu'un joueur attribue à une question ; elle alimente le tri automatique du contenu.

Terme	Définition
Visiteur	Utilisateur non authentifié, jouant en tant qu'invité, sans inscription.
Utilisateur authentifié	Utilisateur disposant d'un compte, qui accède aux fonctionnalités persistantes (profil, statistiques, soumission de questions).
Administrateur	Acteur responsable de la gestion de la plateforme et de la modération des questions soumises.
Modération	Processus de vérification des questions soumises par la communauté avant leur publication.
Scénario nominal	Déroulement standard d'un cas d'utilisation, lorsque tout se passe comme prévu, sans erreur ni cas particulier.
Scénario alternatif / d'erreur	Variante du scénario nominal correspondant à un cas particulier (alternatif) ou à une situation d'échec (erreur).
Incrément	Ensemble cohérent de fonctionnalités livré au cours d'une étape du développement itératif et incrémental.
Temps réel	Mode de fonctionnement où les informations sont échangées et affichées quasi instantanément, sans rafraîchissement manuel, condition essentielle d'une partie multijoueur synchrone.
WebSocket	Technologie de communication établissant une connexion persistante et bidirectionnelle entre le navigateur et le serveur, permettant à ce dernier d'envoyer des données au client sans attendre de requête.
Latence	Délai entre l'émission d'une information et sa réception ; une latence faible est déterminante pour l'équité du jeu.
SGBDR	Système de gestion de base de données relationnelle : logiciel organisant les données en tables reliées par des clés, adapté à des données structurées (par exemple PostgreSQL).
Redis	Magasin de données en mémoire, utilisé ici pour l'état éphémère des parties et comme mécanisme de publication-souscription entre instances du serveur.
Table de jointure	Table intermédiaire qui matérialise, au niveau relationnel, une association plusieurs-à-plusieurs entre deux entités (par exemple entre question et tag).

Terme	Définition
Objet de Jacobson	Catégorisation des objets d'analyse en trois rôles : interface (<i>boundary</i>), contrôle (<i>control</i>) et entité (<i>entity</i>).
OCL	<i>Object Constraint Language</i> : langage de la norme UML servant à exprimer des contraintes formelles sur un modèle. Les contraintes du présent document sont notées entre accolades, à la manière d'UML 1.4.
RGPD	Règlement général sur la protection des données : cadre légal européen encadrant le traitement des données personnelles.

16 Annexe : diagramme de classes en pleine page

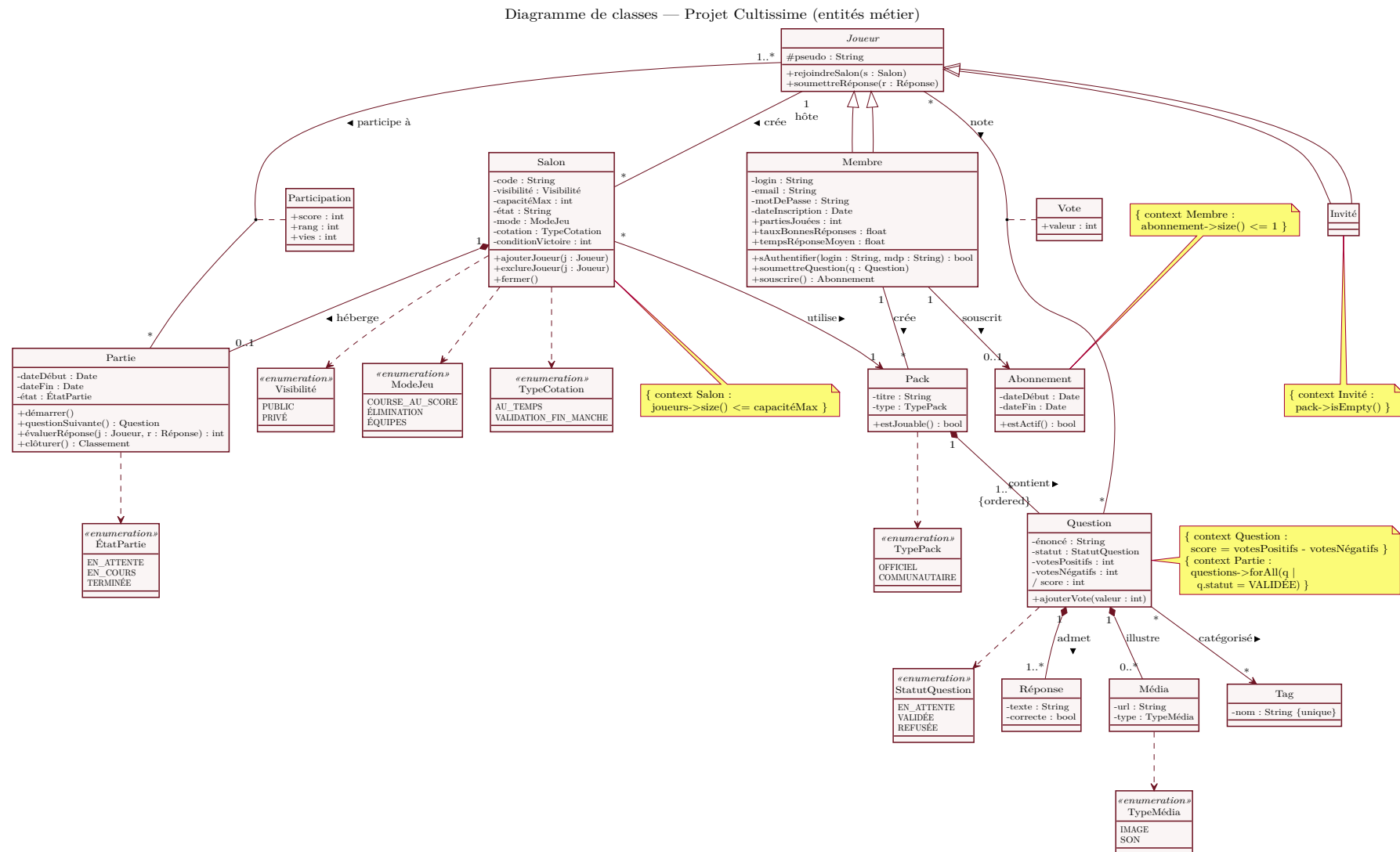


Fig. 54. – Diagramme de classes de conception — version agrandie en pleine page.